

第 4 章

分支预测

4.1 概述

在处理器中,除了 Cache 之外,另一个重要的内容就是分支预测(branch prediction),它和 Cache 一起左右着处理器的性能,一个准确度很高的分支预测器(branch predictor)是提高处理器性能的关键部件。但是在现实世界中,准确度高意味着更复杂的算法,也就是占用更多的硅片面积和消耗更多的功耗,同时,还会影响处理器的周期时间。更不幸的是,不同的程序会呈现不同的特性,因此很难找到一种放之四海而皆准的分支预测算法,一种分支预测算法对某个程序有着很高的预测准确度,但是对另一个程序的分支预测却可能很难如意,这也给分支预测的实现带来了一定的难度。

对于超标量处理器来说,准确度高的分支预测更为重要,在取指令阶段,除了需要从 I-Cache 中取出多条指令,同时还需要决定下个周期取指令的地址,如果这个阶段只是简单地顺序取指令,也就是预测所有的分支指令都是不执行的,那么等到在流水线的后续阶段,例如执行阶段,发现了一条可以执行的分支指令时,就需要将流水线中,执行阶段之前的全部指令都从流水线中清除,并重新从正确的地址开始取指令。这些从流水线中被抹掉的指令都做了无用功,浪费了处理器的功耗,并且降低了执行效率。如果能够在取指令阶段,就可以“预知”本周期所取出的指令中是否存在分支指令,并且可以知道它的方向(跳转或者不跳转),以及目标地址(target address)的话,那么就可以在下个周期从分支指令的目标地址开始取指令,这样就不会对流水线产生影响,也就避免了做无用功,提高了处理器的执行效率。这种不用等到分支指令的结果真的被计算出来,而是提前就预测结果的过程就是分支预测。分支预测之所以能够实现,是由分支指令的特性决定的,因为分支预测本质上是对分支指令的结果进行预测,而在一般的 RISC 指令集中,分支指令包含两个要素。

(1) 方向,对于一个分支指令来说,它的方向只可能有两种,一种是发生跳转(这称为 taken),另一种是不发生跳转(称为 not taken)。有些分支指令是无条件执行的,例如 MIPS 中的 jump 指令,它的方向总是发生跳转的,而对于其他的分支指令,则需要根据指令中携带的条件是否成立来决定是不是发生跳转,例如 MIPS 中的 BEQ 指令,只有当指定的两个值相等的时候,才会发生跳转。

(2) 目标地址,如果分支指令的方向是发生跳转,就需要知道它跳到哪里,也就是它跳转的目标地址,这个目标地址也是携带在指令中,一般来说,对于 RISC 指令集来说,目标地址在指令中可以有两种存在形式。

① PC relative, 也称做直接跳转(direct)。在指令中直接以立即数的形式给出一个相对于 PC 的偏移值(offset), 当前分支指令的 PC 值(或者分支指令的下一条指令的 PC 值)加上这个偏移值就可以得到分支指令的目标地址。由于指令的长度只有 32 位, 它限制了立即数的大小范围, 因此这种类型的分支指令, 它的跳转范围一般不大, 但是由于需要的信息直接携带在指令中, 这样就很容易计算它的目标地址, 例如在流水线的解码(decode)阶段就可以将指令中的立即数分离出来, 进而计算出分支指令的目标地址。而且, 由于指令所携带的立即数一般是不会变化的, 因此这种类型的分支指令是容易进行分支预测的, 很多处理器的技术手册上都会建议尽量使用这种类型的分支指令, 就是为了提高分支预测的正确率, 从而提高处理器的性能。

② Absolute, 也称做间接跳转(indirect)。分支指令的目标地址来自于一个通用寄存器的值, 这个寄存器的编号由指令给出, 这就是说, 它的目标地址是 32 位的值, 因此可以跳转到处理器程序空间中的任意地方。但是, 这个通用寄存器的值一般来自于其他指令的结果, 因此对于分支指令来说, 可能需要等待一段时间才可以得到这个目标地址, 例如需要等到流水线的执行(execute)阶段, 在这段时间内进入到流水线中的指令都是有可能不正确的, 这就增大了分支预测失败时的惩罚(misprediction penalty)。而且, 由于寄存器的值是经常会变化的, 因此这种类型的分支指令很难对目标地址进行预测, 但是庆幸的是, 程序当中大部分间接跳转的分支指令都是用来调用子程序的 CALL/Return 类型的指令, 而这种类型的指令由于有着很强的规律性, 是容易被预测的。除了 CALL/Return 指令之外, 一般的处理器都不会推荐使用间接跳转类型的分支指令。

由于分支指令有着上述两方面的特性, 要对一条分支指令进行预测, 就需要对它的方向和目标地址都进行预测。对于方向预测来说, 需要预测这条分支指令是否会发生跳转; 对于目标地址的预测来说, 需要预测这条分支指令在发生跳转时的目标地址。对于普通的处理器来说, 由于它的流水线深度并不深, 一般都是使用静态的分支预测方式, 预测分支指令总是不执行, 处理器总是顺序地取指令, 在流水线的后续阶段, 例如执行(execute)阶段, 得到了分支指令实际的方向和目标地址后, 再进行判断。如果分支指令需要发生跳转, 则抛弃在分支指令之后进入到流水线的后续指令(或者在 MIPS 中, 在这些流水段放上不相关的指令, 即 branch delay slot); 如果分支指令不需要发生跳转, 则继续顺序地取指令来执行, 就好像这条分支指令从来都没有发生过一样。

对于流水线比较短的简单处理器, 分支预测失败时并不会引起流水线中过多的指令被抛弃, 例如 MIPS R3000 使用了一个五级的流水线, 在流水线的第二个阶段, 即解码(decode)阶段, 就可以得到分支指令的结果, 此时即使分支指令需要发生跳转(因为在取指令阶段都认为分支指令是不发生跳转的, 此时发现它要发生跳转, 这也就是分支预测失败了), 流水线中只有取指令(fetch)阶段的一条指令需要被抛弃, 如图 4.1 所示。

而且, MIPS 处理器为了减少这一个周期的浪费, 在分支指令之后会放置一条不相关的指令, 这条指令总是会被执行, 而不管分支指令是否发生跳转, 分支指令之后的那个位置就被 MIPS 称为分支延迟槽(branch delay slot), 需要编译器(compiler)或者程序员从分支指令之前的程序中找到不相关的指令放到延迟槽的位置。由于这里只需要一条指令, 而从程序中找到一条不相关的指令还是比较容易的。后来随着处理器并行度的提高以及流水线的加深, 当发现分支指令预测失败的时候, 流水线中有很多的指令都错误地进入了流水线中,

它们需要被抛弃,此时即使采用延迟槽的方法,也很难得到满意的结果,因为已经无法从分支指令之前的程序中找到这么多的不相关指令了。很明显,这种只是预测分支指令不发生跳转的静态分支预测算法已经不能够满足复杂处理器对性能的要求,需要更准确的分支预测方法,能够根据处理器实际的执行情况,动态地对分支指令进行预测,这就是动态的分支预测。动态分支预测并不会简单地预测分支指令一直发生跳转或者不跳转,而是会根据分支指令在过去一段时间的执行情况来决定预测的结果,这也是为什么将它称为“动态”的原因。动态分支预测需要消耗不菲的硬件资源,在早期处理器所处的那个年代,由于硅片的面积很昂贵,所以一般不会使用硬件来实现动态分支预测算法。随着晶体管尺寸的缩小,越来越多的功能可以在硅片上实现,这样才使处理器使用动态分支预测的方法成为了可能。如果没有特殊说明,后文中讲述的分支预测算法都是指动态分支预测,“分支预测”和“动态分支预测”这两种称呼是同义的。

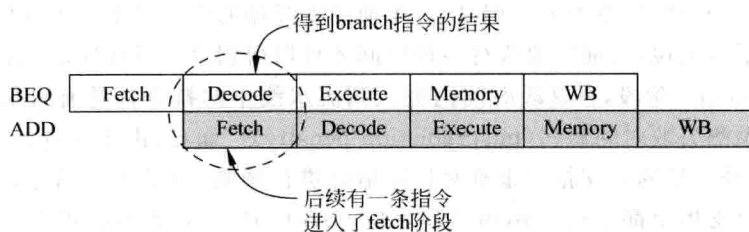


图 4.1 在 MIPS 经典的五级流水线中,分支指令只会影响它后面的一条指令

要进行分支预测,首先需要知道从 I-Cache 取出来的指令中,哪条指令是分支指令,这对于每周期取出多条指令的超标量处理器来说,更为不容易,需要从指令组(fetch group)中找出分支指令,如图 4.2 所示。

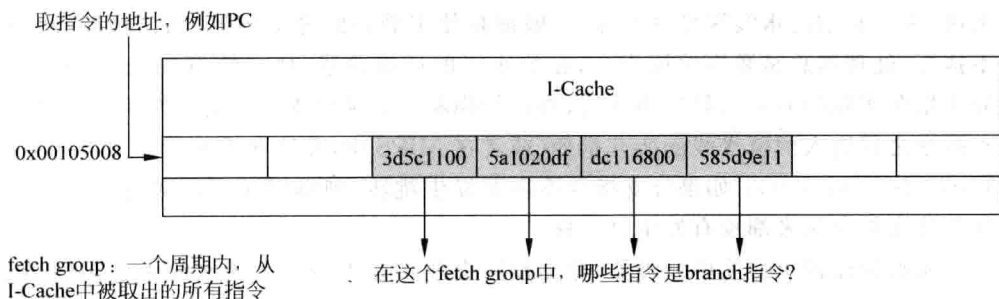


图 4.2 如何才能知道哪条指令是分支指令

最容易想到的方法就是将指令组中的指令从 I-Cache 取出来之后,进行快速的解码,之所以称为快速,是因为只需要辨别解码指令是否是分支指令,然后将找到的分支指令对应的 PC 值送到分支预测器(branch predictor)中,就可以对分支指令进行预测了,如图 4.3 所示表示了这个过程。

当处理器的周期时间比较小时,I-Cache 的访问可能需要多个周期才可以完成,采用图 4.3 所示的方式进行分支预测,从开始取指令直到分支预测得到结果,中间需要间隔好几个周期,在这些周期内无法得到准确的预测结果,只能顺序地取指令,也就相当于这些周期都是预测分支指令不发生跳转,这样就降低了分支预测的准确度,造成了处理器性能的降

低。而且图 4.3 中,指令快速解码(fast decode)和分支预测的过程都放在了同一个周期,严重影响了处理器的周期时间。为了解决这个问题,可以在指令从 L2 Cache 写入到 I-Cache 之前进行快速解码,这也被称为预解码(pre-decode),然后指令是否是分支指令的信息和指令一起写到 I-Cache 中,这样虽然会使 I-Cache 占用更多的面积,但是可以省掉图 4.3 中的快速解码电路,在一定程度上缓解对处理器周期时间的影响,但是,取指令直到分支预测得到结果这两个阶段的间隔时间仍然是过长的,无法得到解决。

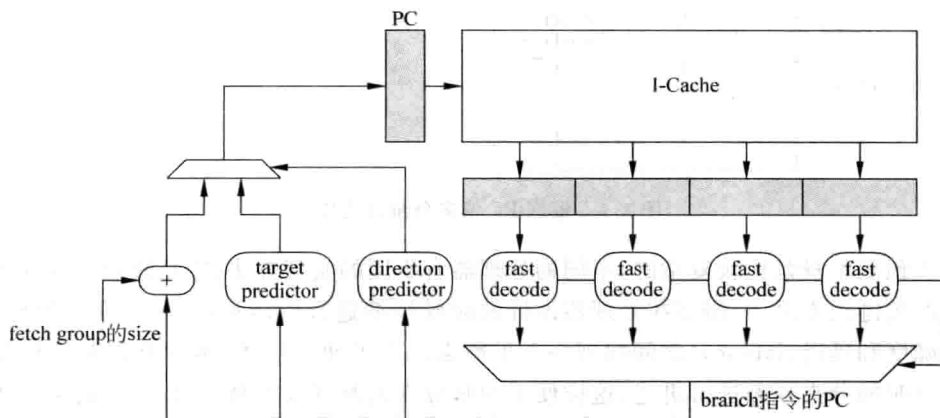


图 4.3 使用快速解码的方式来分辨分支指令

在流水线中,分支预测是越靠前越好的,如果指令从 I-Cache 取出来之后才进行分支预测,如图 4.3 所示,那么由于 I-Cache 中取出指令的过程可能需要多于一个周期才能够完成,当得到分支预测结果时,已经有很多后续的指令进入了流水线,当得到的预测值是要发生跳转时,这些指令都需要从流水线中被抹掉(flush),这样就降低了处理器的执行效率。因此分支预测的最好时机就是在当前周期得到取指令地址的时候,在取指令的同时进行分支预测,这样在下个周期就可以根据预测的结果继续取指令。对于一条指令来说,它的物理地址是会变化的(这取决于操作系统将它放到物理内存的位置),而它的虚拟地址,也就是 PC 值,是不会变化的。因为在一个进程内,每一个 PC 值对应的指令是固定的,不可能出现一个 PC 值对应多条指令的情况,所以使用 PC 值进行分支预测,只不过在进行进程切换之后,需要将分支预测器中的内容进行清空,这样可以保证不同进程之间的分支预测不会互相干扰。如果使用了 ASID,那么可以将它和 PC 值一起进行分支预测,此时就不需要在进程切换时清空分支预测器了。

在 PC 值刚刚产生的那个周期,根据这个 PC 值来预测本周期的指令组(fetch group)中是否存在分支指令,以及分支指令的方向和目标地址,这个过程如图 4.4 所示。

基于取指令的地址(也就是取指令的 PC 值)进行分支预测是有根据的,因为一旦程序开始执行,每条指令对应的取指令地址也就固定好了,因此完全可以根据一条指令的 PC 值来判断这条指令是否是分支指令。只要这条分支指令第一次被执行完之后,当后面再次遇到这个 PC 值,就可以知道当前要取的指令是分支指令。即使发生了自修改(self-modifying)的情况,也会将分支预测器清空,重新开始进行分支预测,并不会影响分支预测的过程。当然,这仅仅是能够识别出它是分支指令,对于它的方向和目标地址,还需要其他的预测方法,这就是本章要重点讲述的内容。

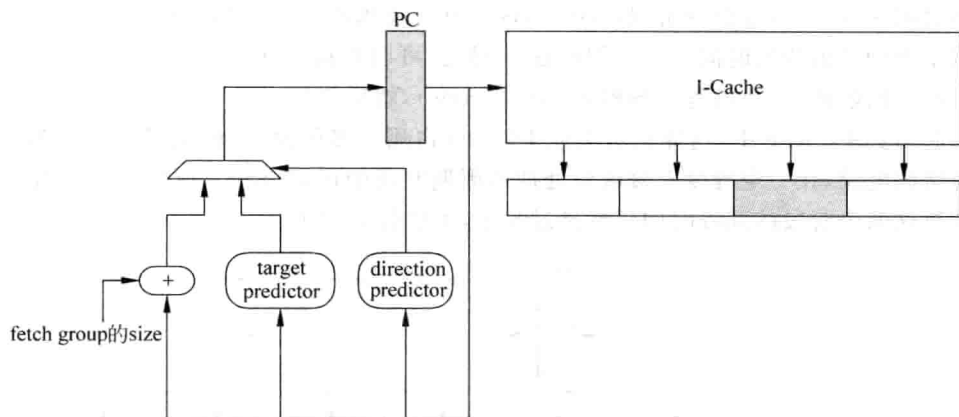


图 4.4 根据 PC 值来分辨分支指令

分支预测本身是比较复杂的,不同的处理器有不同的实现方法,它是影响一款处理器性能高低的关键因素之一,需要在处理器设计的时候仔细进行权衡和折中,以便在硬件消耗、预测准确度和延迟(latency)之间找到一个平衡点。为了便于讲述,本章首先对每周期只取一条指令时的分支预测进行讲述,这样便于理解分支预测算法本身的内容,然后再讲述超标量处理器中使用的分支预测算法,由于超标量处理器每周期会取出多条指令,这会给分支预测带来一些挑战,不同的处理方法会导致不同的硬件消耗和性能表现,这些内容都会在本章进行讲述。

4.2 分支指令的方向预测

对于分支指令来说,它的方向只有两个:发生跳转(taken)和不发生跳转(not taken),因此可以用 1 和 0 来表示。而且,很多分支指令的方向是有规律可循的,考虑下面的一个例子。

```
for ( i = 0; i < 1000; i++)
{ 循环体 }
```

上面这个 for 语句,会执行 1000 次循环体的内容,这个 for 语句会被编译器变成一条分支指令,这个分支指令会向同一个方向执行 1000 次(例如执行 1000 次跳转,最后一次是不跳转),因此对于这条分支指令来说,它的方向是有规律的。

动态分支预测的一个主要方面就是对分支指令的方向进行预测,使用的方法可繁可简,最简单的动态预测方法是直接使用上次分支的结果,这种方法也称为根据最后一次的结果进行预测(last-outcome prediction),这种方法如图 4.5 所示。

如图 4.5 所示的方法相比于静态分支预测,在很多情况下都可以获得更优的结果,但是当分支指令的方向发生变化时,仍旧会引起分支预测失败(mis-prediction),如图 4.6 所示。

在图 4.6 所示中,分支指令每执行 10000 次,就会发生两次分支预测失败,因此分支预测的失败率是 $2/10000 = 0.002\%$,看起来似乎已经很不错了,但是一条分支指令有着如图 4.7 所示的执行情况时呢?

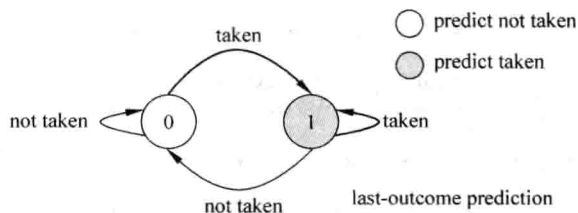


图 4.5 根据分支指令最后一次的结果进行预测

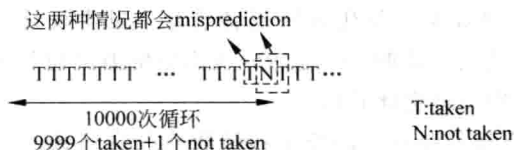


图 4.6 当分支指令的方向发生变化时,就会预测错误

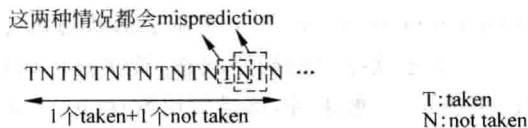


图 4.7 方向变化比较频繁的分指令

由于图 4.7 中的分支指令的方向每次都发生变化,则使用最后一次执行的结果进行分支预测的方法,它的失败率将是 100%! 此时预测准确度反而比不上静态分支预测的方法,因为对于图 4.7 所示的例子来说,静态分支预测的方法是 50% 的失败率,很显然,这种使用最后一次结果进行分支预测的方法,它的准确度是无法接受的,因此它并没有在现代处理器中被采纳。现代处理器中应用最广泛的分支预测方法都是基于两位饱和计数器(2-bit saturating counter),并以之为基础,引申出的各种分支预测的方法。

4.2.1 基于两位饱和计数器的分支预测

基于两位饱和计数器的分支预测方法并不会马上使用分支指令上次的结果,而是根据一条分支指令前两次执行的结果来预测本次的方向,这种方法可以用一个有着 4 个状态的状态机来表示,这四个状态分别如下。

(1) Strongly taken: 计数器处于饱和状态,分支指令本次会被预测发生跳转(taken),编码为 11;

(2) Weakly taken: 计数器处于不饱和状态,分支指令本次会被预测发生跳转(taken),编码为 10;

(3) Weakly not taken: 计数器处于不饱和状态,分支指令本次会被预测不发生跳转(not taken),编码为 01;

(4) Strongly not taken: 计数器处于饱和状态,分支指令本次会被预测不发生跳转(not taken),编码为 00。

整个状态机如图 4.8 所示。

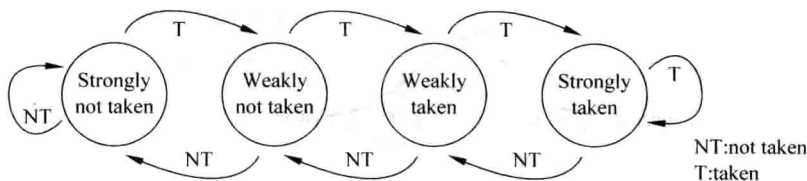


图 4.8 基于两位饱和计数器的分支预测

状态机处于饱和状态时,只有两次预测失败才会改变预测的结果。当分支指令的方向总是朝着同一个方向时,例如总是发生跳转或者总是不发生跳转,状态机就会处于饱和状态,此时预测的正确率就是比较高的;但是当分支指令的方向总是变化时,状态机就无法处于饱和状态,此时预测正确率就会比较低。

如图 4.8 所示的状态机只是基于两位饱和计数器的众多预测方法中的一种,这种实现方式是应用最广泛的。事实上,这个状态机有其他的很多种实现方式,下面给出了其他两种可能的方式。

对于一般的基准测试(benchmark)程序,图 4.9 所示的状态机并不会取得更优的结果,因此它们的使用并不广泛。而对于状态机的初始状态,其实是可以位于四个状态中的任意一个,需要根据实际的情况来决定,一般来讲,都是使用 Strongly not taken 或者 Weakly not taken 作为初始态。

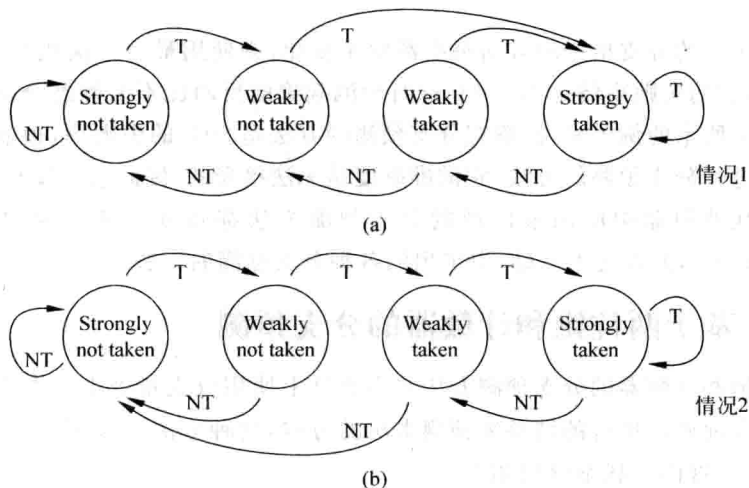


图 4.9 基于两位饱和计数器的另外两种预测方法

可以使用格雷码对状态机进行编码,保证在状态转换时每次只有一位发生变化,这样可以减少出错的概率,并降低功耗。在两个饱和的状态(Strongly not taken 或者 Strongly taken),计数器达到了最大值或最小值,此时分支指令的方向如果继续保持不变时,就会使状态停留在原地,不会再发生变化,相当于“饱和”了,这也是饱和计数器名称的由来。基于两位饱和计数器的预测方法,它的正确率要优于之前讲述的基于上一次结果的分支预测方法,例如,仍旧考虑下面的 C 代码。

```
for( i = 0; i < m; i++)
```

{循环体}

当这个 for 循环第一次执行时,状态机处于 Weakly not taken 状态(假设状态机复位时处于此状态),因此会预测分支指令是不发生跳转的,显然这个预测是错误的,因此 for 循环的第一次会发生预测失败,这会使状态机跳转到 Weakly taken 状态(最后会稳定在 Strongly taken 状态),因此在 for 循环的第二次直到第 $m-1$ 次,分支预测都是正确的。在 for 循环的最后一次会产生一次错误的预测,因为此时应该不发生跳转了,而状态机会预测发生跳转(此时状态机处于饱和的 Strongly taken 状态),这会使状态机跳转到 Weakly taken 状态。因此,在第一次执行整个 for 循环时,产生了两次预测错误,出错的概率为 $2/m$,如果 m 值很大,则两次的出错是可以忍受的。

当再次执行这个 for 循环体时,for 循环的第一次执行就会预测正确了,因为此时的状态机处于 Weakly taken 状态,仍旧是预测发生跳转的,这会使状态机跳转到饱和的 Strongly taken 状态,故 for 循环体的第二次到第 $m-1$ 次也仍旧是预测正确的,只有 for 循环的最后一次才发生了预测错误,此时出错的概率降为 $1/m$ 。

由上面的执行过程可以看出,使用基于两位饱和计数器的分支预测,核心理念就是当一条分支指令连续两次执行的方向都一样时,那么该分支指令在第三次执行时也会有同样的方向;如果一条分支指令只是偶尔发生了一次方向的改变,那么这条分支指令的预测值不会马上跟着改变,因此这种方式的分支预测就好像是有一定时间的延迟一样,分支方向偶尔的变化将会被过滤掉。

对于绝大多数程序来说,使用这种基于两位饱和计数器的分支预测方法,就已经可以获得比较高的正确率了,增加计数器的位宽(例如使用三位的计数器)会引起复杂度的上升和更多存储资源的需求,这些额外开销所带来的负面影响要大于实际可以获得的预测精度的提高,因此主流的处理器的处理器一般都是基于两位的饱和计数器来实现分支预测的。

在前面说过,分支预测都是以 PC 值为基础进行的,正常来说,每一个 PC 值都应该对应一个两位的饱和计数器(也就是每条指令都对应一个饱和计数器)。因此,对于 32 位的 PC 值来说,共需要 $2^{30} \times 2b$ 大小的存储器来存储这些计数器的值(指令是字对齐,所以不考虑 PC 值的最低两位),显然实际芯片当中无法使用这样大的存储器。考虑到并不是所有的指令都是分支指令,一般使用图 4.10 所示的方法来存储两位饱和计数器的值^[15]。

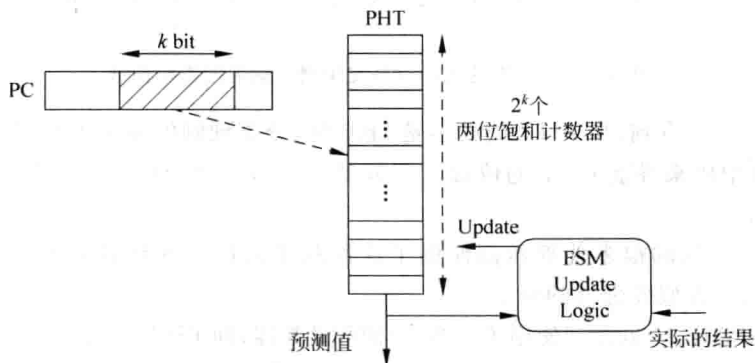


图 4.10 使用 PC 值的一部分来寻址饱和计数器

在图 4.10 中, PHT(Pattern History Table) 是一个表格, 在其中存放着所有 PC 值(此处是 PC 值的一部分)对应的两位饱和计数器的值, 这个 PHT 使用 PC 值的一部分来寻址, 例如图 4.10 中使用 PC 值当中的 k 位来寻址 PHT, 因此 PHT 的大小就是 $2^k \times 2\text{bit}$, k 值越大, PHT 就可以记录更多 PC 值对应的饱和计数器的值, PHT 所占用的存储空间也就更大。但是, 使用这种方式来寻址 PHT, 必然就导致了 k 部分相同的所有 PC 值都使用同一个两位饱和计数器的值, 如果这些 PC 值对应的指令中不止有一条分支指令, 那么相互之间肯定会产生干扰, 这种情况称为别名(aliasing)。别名的存在会对分支预测的准确度产生影响, 如果两条别名的分支指令的方向是相同的, 例如都会发生跳转, 那么它们对应的两位饱和计数器就会保持在饱和状态, 此时不会对这两条分支指令的预测准确度产生负面的影响, 这种情况称为中立的别名(neutral aliasing); 如果两条别名的分支指令的方向不同, 那么它们对应的两位饱和计数器就会一直无法处于饱和状态, 当然就降低了分支预测的准确度, 这种情况称为破坏性的别名(destructive aliasing)。虽然图 4.10 所示的这种截取 PC 值的一部分来寻址 PHT 的方法存在别名的问题, 从而可能会导致分支预测准确度的降低, 但是考虑到这个方法实现起来比较容易, 而且占用的存储器资源也不大, 因此轻微的预测准确度降低也是能够接受的。图 4.10 中所示的方法就是基于两位饱和计数器的分支预测方法, 也称为双峰分支预测器(bimodal predictor), 因为这种方法有两个饱和的状态, 在这种方法中, 饱和计数器的多少(也就是 PHT 的大小)直接影响着分支预测的准确度, 根据参考文献^[16], PHT 的大小对分支预测准确度的影响如图 4.11 所示。

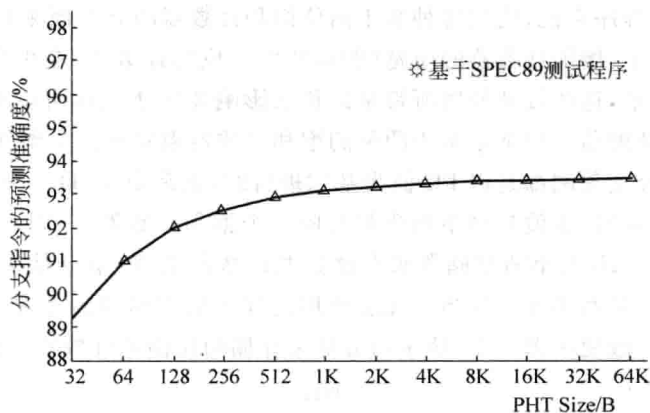


图 4.11 PHT 的大小对分支预测准确度的影响曲线

从图 4.11 可以看到, 当 PHT 的大小是 2KB 时, 分支预测的准确度已经达到了 93% 以上, 此时 PC 值中用来寻址 PHT 的位数 $k = \log(2 \times 1024 \times 8\text{b}/2\text{b}) = 13$, 即使用 PC 中的 13 位来寻址 PHT。

20 世纪 90 年代的很多处理器都使用了这种基于两位饱和计数器的方法进行分支预测, 下面给出几个典型处理器的例子。

- (1) Sun SPARC64 处理器使用了 1024 个饱和计数器, 即 PHT 的大小是 $1\text{K} \times 2\text{b} = 2\text{Kb}$ 。
- (2) MIPS R10K 处理器使用了 512 个饱和计数器, 即 PHT 的大小是 $512 \times 2\text{b} = 1\text{Kb}$ 。
- (3) IBM PowerPC 620 处理器使用了 2048 个饱和计数器, 即 PHT 的大小是 $2\text{K} \times 2\text{b} = 4\text{Kb}$ 。

(4) Intel Pentium 处理器使用了 512 个饱和计数器,不过它将 PHT 和 BTB 集成在了一起,因此需要使用 Cache 的模型来实现 PHT,这对 PHT 的预测准确度会有影响,关于 BTB 的内容会在本章的后面进行介绍。

在基于两位饱和计数器的分支预测方法中,别名(aliasing)会降低分支预测的准确度,可以采用一些更高级的方法来避免别名情况的发生,比较典型的方法就是使用哈希(Hash),对 PC 值进行处理之后再去寻址 PHT,如图 4.12 所示。

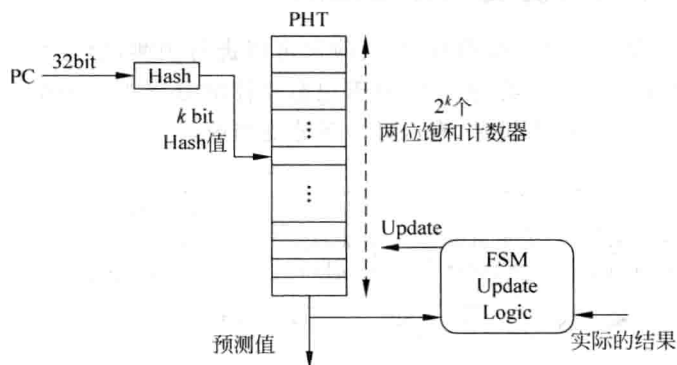


图 4.12 对 PC 值进行处理之后,再去寻址 PHT

使用这种方法,可以避免直接使用 PC 值中的一部分来寻址 PHT 而引起的别名问题,图 4.12 中的哈希(Hash)算法能够将 32 位的 PC 值压缩为固定长度的较小的值,这个小的值称为哈希值,在计算机算法领域中,哈希指的就是把任意长度的输入值,通过哈希算法,变为固定长度的输出,一般输出值的长度远小于输入值的长度。哈希算法的实现可以很简单,例如使用普通的异或逻辑,当然为了得到好的效果,也可以做得很复杂,不同的公司、不同的处理器会有不同的实现策略,这部分内容本书不作讨论。

两位的饱和计数器需要根据分支指令的结果(发生跳转或者不发生跳转)进行更新,有三个时间点可以对 PHT 进行更新。

- (1) 在流水线的取指令阶段,进行分支预测时,根据预测的结果来更新 PHT;
- (2) 在流水线的执行阶段,当分支指令的方向被实际计算出来时,更新 PHT;
- (3) 在流水线的提交(Commit)阶段,当分支指令要离开流水线时,更新 PHT。

第(1)种方法对基于两位饱和计数器的分支预测方法是不适用的,因为此时分支预测的结果可能是错误的,使用错误的结果来更新 PHT 是不可靠的。后两种方法在流水线的执行阶段和提交阶段更新 PHT,但是因为超标量处理器有着很深的流水线和每周周期执行好几条指令的这两个特点,导致一条分支指令可能在 PHT 更新之前就被取过很多次了(例如 for 循环体很短的代码),这条分支指令在进行分支预测的时候,并没有利用到它之前执行的分支结果。但是考虑到这种饱和计数器的特点,只要计数器处于饱和状态,它的预测值就会比较固定,即使更新 PHT 的时间晚一些,也不会改变计数器的饱和状态,所以并不会对分支预测的精度产生太大的负面影响。虽然第(2)种方法更新 PHT 的时间早于第(3)种方法,但是在超标量处理器中,如果对分支指令采用了乱序执行(out-of-order)的方式,那么即使在执行阶段得到了一条分支指令的结果,也不能够保证这个结果是正确的,因为这条分支指令可能处于分支预测失败(mis-prediction)的路径上。也就是说,它之前有分支指令预测

一个寄存器来记录一条分支指令在过去的历史状态,当这个历史状态很有规律时,就可以为分支预测提供了一个可以利用的工具,这样的寄存器称为分支历史寄存器(Branch History Register, BHR),这种预测方法称为基于局部历史(Local History)的预测方法。对一条分支指令来说,通过将它每次的结果(发生跳转或者不发生跳转,用1或0表示)移入 BHR 寄存器,就可以记录这条分支指令的历史状态了,如果这条分支指令很有规律,那么就可以使用 BHR 寄存器对这条分支指令进行预测,这种分支预测方法的工作机制如图 4.14 所示。

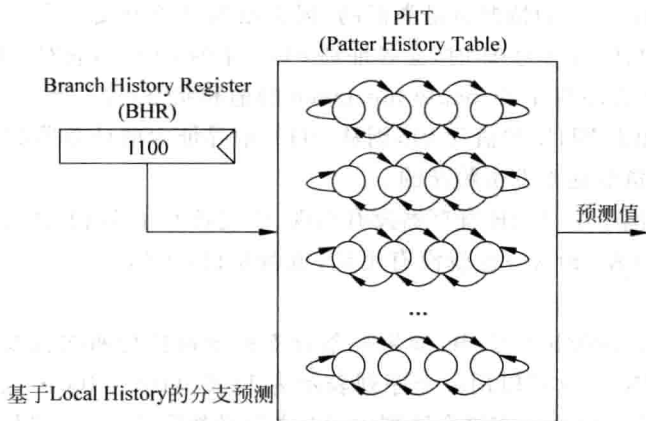


图 4.14 基于局部历史的分支预测

这种方法也称为自适应的两级分支预测(Adaptive Two-level Predictor),一个位宽为 n 位的 BHR 寄存器可以记录一条分支指令过去 n 次的结果(即发生跳转或不发生跳转),对 BHR 使用一个两位的饱和计数器来捕捉它的规律,因此需要使用 BHR 去寻址 PHT (Pattern History Table),此处的 PHT 仍旧是一个表格,它的大小为 $2^n \times 2\text{bit}$,PHT 当中存有 BHR 的每种取值对应的两位饱和计数器的值。虽然在图 4.14 中,PHT 中的每个表项(entry)都似乎有一个两位计数器,其实不是这样的,在其中只是存储了饱和计数器的值,计数器的更新独立于 PHT,这和上一节讲过的内容是一样的,只需要对整个 PHT 使用一个对饱和计数器进行更新的硬件就可以了。当一条分支指令得到结果时,就将 PHT 中对应的计数器值读出来,并根据分支结果对其进行更新,然后将这个结果写回到 PHT 中即可,这样就相当于 PHT 当中的每个表项都有一个饱和计数器,如图 4.14 所示的那样。PHT 当中的两位饱和计数器试图寻找出 BHR 的规律,如果一条分支指令过去的行为很有规律,即 BHR 中的值很有规律的话,用 PHT 当中的两位饱和计数器就能很好地捕捉到这种规律。理论上讲,每条分支指令都有一个 BHR 寄存器和一个 PHT,下面通过例子来说明图 4.14 是如何工作的。

假设有一条分支指令,它的 BHR 寄存器的宽度是两位,意味着这条分支指令前两次的结果会被保存在 BHR 寄存器中。BHR 有四种不同的值 00、01、10 和 11,其中 0 表示没有发生跳转,1 表示发生了跳转,这时需要一个有着 4 个表项(entry)的 PHT,用来和 BHR 的四种取值一一对应,称为 entry0、entry1、entry2 和 entry3。假设这条分支指令有如下的执行顺序 taken→not taken→taken→not taken→taken...,这可以表示为 10_01_10...,分支指令的执行结果从 BHR 寄存器右边移入,则在 BHR 寄存器中会交替出现 10 和 01,即 BHR

中的值是 10 时,下次移入的值肯定是 1,也就是这条分支指令肯定下次是要跳转的。同理,BHR 中的值是 01 时,之后肯定会移入 0,也就是这条分支指令下次肯定不会发生跳转;BHR 当中的值是 10 时,会寻址到 PHT 当中的 entry2,对于这个表项中的饱和计数器来说,因为每次都会发生跳转,所以这个计数器会到达 Strongly taken 的饱和状态,这样当这条分支指令进行分支预测时,如果发现它的 BHR 值为 10,从 PHT 中寻址到的计数器值就是 11,也就可以预测出这条分支指令会发生跳转。

对于 BHR 的值是 01 的情况也是类似的,因为根据这条分支指令的执行情况,01 之后肯定会跟着 0,而 BHR 的值为 01 时,会寻址到 PHT 中的 entry1,它对应的计数器因为总是不发生跳转,则肯定会停留在 Strongly not taken 的饱和状态,这样当这条分支指令在进行分支预测的时候,如果 BHR 的值为 01,则从 PHT 中寻址到的计数器的值就是 00,也就可以预测到这条分支指令是不发生跳转的。

在上面的这个例子中,BHR 寄存器没有出现 00 或者 11 的取值,因此 PHT 当中对应的表项(也就是 entry0 和 entry3)也就没有用到,也就是说,PHT 中的四个表项,只有两个是实际起作用的。

上面的例子可以继续引申,如果一条分支指令每执行两次就改变分支的方向,即 TTNNTTNNTTN...,这可以用一个序列表示为 1100_1100_1100...,这种方式仍可以用上面两位的 BHR 进行预测。在这个序列中,11 之后必然跟着 0,10 之后必然跟着 0,00 之后必然跟着 1,01 之后必然跟着 1。也就是说,序列当中每两位的数后面跟着的数值都是唯一的,称这个序列的循环周期为 2,经过一段时间之后,PHT 就可以捕捉到这条分支指令的规律,entry0 和 entry1 中的两个计数器由于输入总是 1,会停留在饱和的 Strongly taken 状态;entry2 和 entry3 中的两个计数器由于输入总是 0,会停留在饱和的 Strongly not taken 状态。使 PHT 中的饱和计数器到达饱和状态的这段时间称为训练时间(training time),在这段时间内,由于计数器没有到达饱和状态,因此分支预测的准确度是比较低的,训练时间的长短取决于 BHR 寄存器的位宽,一个位宽很大的 BHR 寄存器需要更多的时间来找出规律,因此它的训练时间也就比较长,这对于分支预测的准确度是有负面影响的,同时更宽的 BHR 需要 PHT 中有更多的饱和计数器,也就是增加了存储器的占用。但是,更宽的 BHR 可以记录到分支指令更多的历史信息,这可以提高分支预测的准确度,因此 BHR 的位宽究竟要多少位,是需要根据实际的设计进行权衡和折中的。

以上所列举的两条分支指令的执行情况是 TNTNTN...和 TTNNTTN...,使用两位的 BHR 就可以很成功地对其进行分支预测,如果使用更宽的 BHR 来预测就更没有问题了。其实,对于一个最小循环周期为 n 的序列来说,任何大于 n 的值都可以作为这个序列的循环周期。举例来说,序列“110011001100...”中,每相邻的两位(11、10、00 和 01),它们后面都跟着不同的值,故这个序列的循环周期为 2,那么这个序列每相邻的三位(110、100、001 和 011)后面也会跟着不同的值,此时也可以称它的循环周期为 3,所以对于循环周期为 1 或者 2 的序列来说,如果使用三位或者更宽的 BHR,仍可以准确地进行分支预测,更不用提更宽的 BHR 了。

但是,如果使用的 BHR 寄存器的宽度,小于分支指令序列的循环周期,例如使用两位的 BHR 去预测序列“000100010001...”就不会那么准确了。因为在序列中,相邻的两位数,例如 00,后面既可能跟着 0,也可能跟着 1,所以在 entry0 中的计数器不能学到一个固定的

规律,也就无法停留在一个固定的饱和状态,此时当然就无法提供准确的预测值了。其实,上面的这个序列的循环周期是 3,因此可以使用宽度不小于三位的 BHR 对其进行分支预测。

通过以上的这些例子,可以总结出一个一般性的规律:如果一个序列中,连续相同的数最多有 p 位,那么这个序列的循环周期就为 p ,例如序列“11000_11000_11000...”,因为有两个连续的 1,三个连续的 0,故循环周期为 3,而不是 2;序列“00000111_00000111...”中,有五个连续的 0,三个连续的 1,故循环周期为 5,而不是 3;只要分支历史寄存器 BHR 的宽度 n 不小于序列的循环周期 p ,那么就可以对该序列进行完美的预测。

下面再用一个例子来加深对上面内容的理解。

例如,如果一个 for 循环的循环次数很小,称为 loop closing,假设 for 循环的循环次数为 4,则这个 for 循环被编译成汇编指令之后,肯定会有一条分支指令,这条分支指令就会有类似于 TTTNTTTN...这样的行为,可以用序列“1110_1110_1110...”来表示,1 表示发生了跳转(taken),0 表示没有发生跳转(not taken)。上面这个序列的循环周期是 3,则任何宽度不小于 3 的 BHR 都可以对其进行预测,现在使用一个四位的 BHR,则 PHT 中有 16 个饱和计数器,其预测过程如图 4.15 所示。

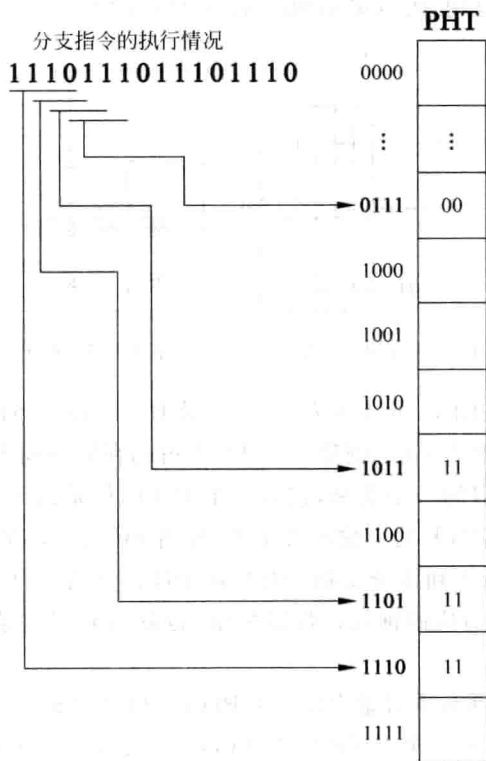


图 4.15 使用基于局部历史的预测算法进行分支预测的一个例子

使用四位的 BHR 记录这个序列之后,这个序列会重复地出现 1110、1101、1011 和 0111,并且规律如下。

1110 之后必然跟着 1,故 PHT 当中 1110 对应的计数器会停在 Strongly taken 的饱和

状态(即 11);

1101 之后必然跟着 1,故 PHT 当中 1101 对应的计数器会停在 Strongly taken 的饱和状态(即 11);

1011 之后必然跟着 1,故 PHT 当中 1011 对应的计数器会停在 Strongly taken 的饱和状态(即 11);

0111 之后必然跟着 0,故 PHT 当中 0111 对应的计数器会停在 Strongly not taken 的饱和状态(即 00)。

通过上述规律可以看出,这条分支指令只使用了 PHT 当中的四个计数器,并没有使用到其他的,因此对于这条分支指令来说,PHT 当中的其他 12 个计数器都是浪费的。

到目前为止,以上讲述的分支预测方法有一个大前提,那就是每条分支指令都有自己对应的 BHR 和 PHT,其中 PHT 的大小是和 BHR 的宽度成指数的关系,一个 n 位的 BHR 需要 PHT 的大小为 $2^n \times 2\text{bit}$ 。如果为每条分支指令都配一个 BHR 和 PHT,这样需要很大的存储空间,将所有分支指令的 BHR 组合在一起称为分支历史寄存器表(Branch History Register Table, BHRT 或 BHT),在实际当中,BHT 不可能照顾到每个 PC,一般都是使用 PC 的一部分来寻址 BHT,这就相当于一些 PC 会共用一个 BHR,同时,PHT 由于要占用大量的存储空间,更需要被复用,可以采用如图 4.16 所示的方法。

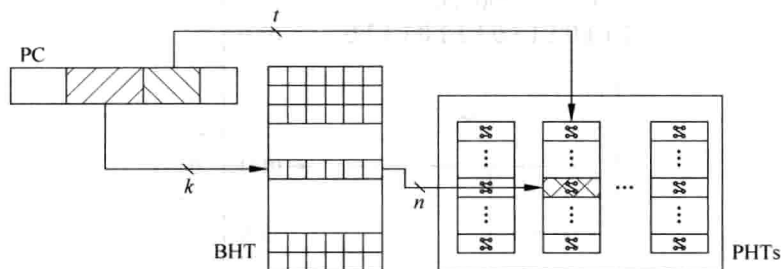


图 4.16 使用 PC 值的一部分来寻址 BHT 和 PHT

使用 PC 的一部分(在图 4.16 中为 k 位)来寻址 BHT,得到当前指令对应的分支历史寄存器 BHR,设 BHR 的宽度为 n 位,则整个 BHT 所占的存储空间为 $2^k \times n\text{bit}$ 。图 4.16 中的 PHTs 表示由 PHT 所组成的一个表格,包含多个 PHT,但是由于 PHT 所占用的存储空间很大,所以 PHTs 中含有 PHT 的个数不会很多,使用 PC 的一部分(图 4.16 中为 t 位,一般来说, $t < k$)来寻址 PHTs,得到该分支指令对应的 PHT,然后用这条分支指令的 BHR 去寻址这个 PHT,就可以得到对应的饱和计数器的值,也就得到了这条分支指令对于方向的预测值了。

由于使用了 PC 的一部分来寻址 BHT 和 PHTs,可能会遇到重名(aliasing)的情况,这些重名的分支指令会使用同一个 BHR 或 PHT,相互之间有了干扰,这样就会使分支预测准确度有所下降。

在图 4.17 中,PHT 占用的存储空间是比较大的,为了更大限度节约 PHTs 所占用的存储空间,一个极端情况是 PHTs 中只含有一个 PHT,如图 4.17 所示。

由于只有一个 PHT,不需要 PC 来寻址 PHT 了,BHT 中的所有 BHR 都寻址这一个 PHT,这样肯定是有冲突的情况发生的,例如两条不同的分支指令对应两个不同的

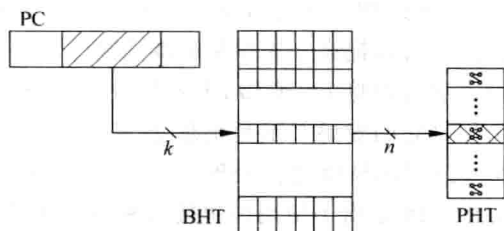


图 4.17 为所有分支指令只使用一个 PHT

BHR,但是 BHR 的内容相同,则此时会对应到 PHT 的同一个计数器,它们之间就产生了干扰,有可能使饱和计数器无法达到饱和的状态,从而影响分支预测的准确度。但是通过上面的例子可以知道,对于一个 BHR 来说,只会用到 PHT 的少部分内容,这样多个 BHR 就可能分别使用 PHT 中不同的部分,可以最大限度地利用 PHT 中的计数器,避免了 PHT 存在过多用不到的内容从而浪费存储空间的情况,但是不可避免的,会存在下面两种冲突的情况。

情况一:两条分支指令的 PC 值对应的 k 部分相同,此时这两条分支指令就会对应到同一个 BHR 寄存器,也就对应到了 PHT 中的同一个饱和计数器,这样,两条分支指令的结果就会互相干扰,降低了分支预测的准确度。

情况二:两条分支指令虽然对应着两个不同的 BHR,但是这两个 BHR 中的内容是一样的,这样两条分支指令也会共用 PHT 当中的同一个饱和计数器,也会降低分支预测的准确度。

为了避免上述两种情况,可以将 PC 值和对应的 BHR 值进行一定的处理,使用处理之后的值来寻址 PHT,如图 4.18 所示。

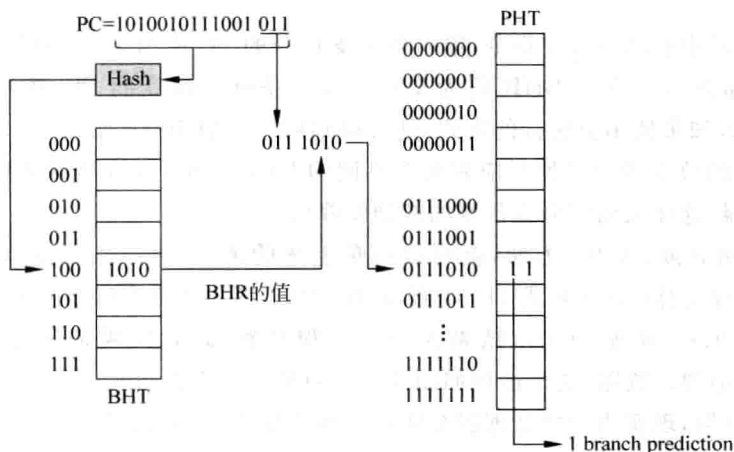


图 4.18 将 PC 值和 BHR 值进行一定的处理

将 PC 值进行哈希处理之后,得到一个固定长度的值,再用此值来寻址 BHT,这样可以解决上面提到的情况一当中遇到的问题。从 BHT 中可以得到这条分支指令对应的 BHR 值,再将该 BHR 的值和 PC 值的一部分进行拼接,用得到的新值来寻址 PHT,从而得到饱和计数器的值,也就是得到了分支预测的结果,这样可以解决上面提到的情况二当中遇到的

问题。综合来看,图 4.18 所示的这种方法将分支指令的 PC 值也考虑进来,一定程度上避免了上述两种冲突的情况,因此可以提高分支预测的准确度。

需要注意的是,有多种方法可以将 PC 值和 BHR 值进行处理,图 4.18 所示的这种位拼接的方式是最简单的方式,但是它的效果并不是很理想,有时候会引入新的冲突,其实还有很多其他的方式可以对 PC 值和 BHR 值进行处理,所获得的效果也会更好一些,例如使用异或(XOR)法,举例来说,图 4.19 给出了使用位拼接法和异或法进行处理时的情况。

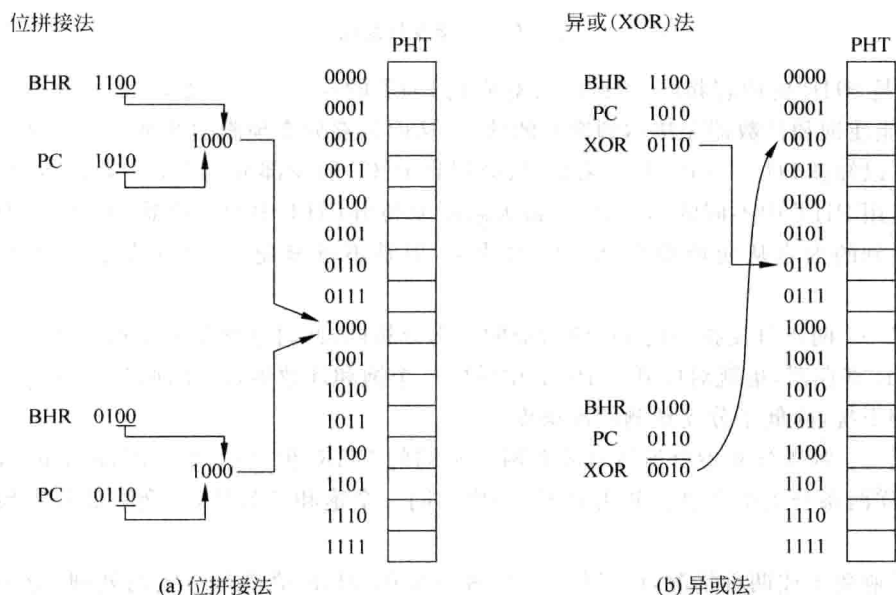


图 4.19 使用位拼接法和异或法进行处理时的情况

图 4.19 所示中有两条分支指令,第一条分支指令的 PC 值为 1010(这四位其实是来自于 PC 值的一部分),对应的 BHR 值为 1100;第二条分支指令的 PC 值为 0110,对应的 BHR 值为 0100,如果使用位拼接的方式,则会得到同一个值 1000,如图 4.19(a)所示,也就是说,两条不同的分支指令即使对应着两个不同的 BHR 的值,也可能寻址到 PHT 中的同一个饱和计数器,这样反而使分支预测的准确度降低了。

而如果使用异或(XOR)方法,就可以避免上述的情况,第一条分支指令的 PC 值和 BHR 值进行异或运算(1010 异或 1100),结果为 0110;第二条分支指令的 PC 值和 BHR 值进行异或运算(0110 异或 0100),结果为 0010。很显然,此时这两条分支指令会寻址到 PHT 中的不同饱和计数器,这个过程如图 4.19(b)所示。由此可见,使用异或方法会获得相对好一些的效果,现实当中的处理器会使用一些更复杂的算法,尽量避免 PHT 冲突的情况发生。

本节所讲述基于 BHR 的分支预测方法,由于只考虑被预测的分支指令自身在过去的执行情况,所以称之为基于局部历史(Local History)的分支预测法。理论上讲,任何一条分支指令,只要它的执行是有规律的,都可以使用这种方法进行分支预测,但是现实的情况并没有那么乐观:当一条分支指令的循环周期太大时,就需要一个宽度很大的 BHR 寄存器,这会导致过长的训练时间(training time),并且 PHT 也会随之占用更多的存储器资源,

这在现实的处理器中都是没有办法接受的。现实世界的处理器只能够使用有限位宽的BHR,这样就会导致,即使对很有规律的分支指令,例如999次跳转加1次不跳转,也无法获得最完美的分支预测准确度,但是这种方法相比基于两位饱和计数器进行分支预测的方法来说,已经是一个很大的进步了,其实从本质上来说,基于两位饱和计数器的分支预测方法也可以称为基于局部历史的预测方法,因为它们都是根据分支指令自身在过去的执行情况来进行预测的,也就是说,它们并没有考虑到一条分支指令之前的其他分支指令对自身预测结果的影响。而有些时候,一条分支指令的结果并不取决于自身在过去的执行情况,而是和它前面的分支指令的结果是息息相关的,这就是4.2.3节要讲述的内容。

4.2.3 基于全局历史的分支预测

如果对一条分支指令进行分支预测时,考虑到它前面的分支指令的执行结果,则称这种预测方法为基于全局历史(Global History)的分支预测。一条分支指令的结果什么时候会和它之前的分支指令相关呢?以SPEC程序当中的一段代码为例。

```
if(aa == 2) /b1/
    aa = 0;
if(bb == 2) /b2/
    bb = 0;
if(aa != bb) /b3/
    { ... }
```

通过观察上面的代码,很容易发现,如果分支b1和b2都执行了,那么分支b3就不会执行,只依靠分支b3的局部历史进行分支预测,是永远不会发现这个规律的,因此需要在预测分支b3的时候将前面的分支b1和b2的结果也考虑进去,这就是基于全局历史(Global History)的分支预测法,也称为全局相关的分支预测。

在4.2.2节讲述的基于局部历史(Local History)的分支预测方法中,每一条分支指令都会有自己对应的BHR寄存器(或者为了成本的考虑,某几条分支指令对应一个BHR寄存器),每个BHR寄存器只是记录与它相对应的分支指令在过去的执行情况;而在基于全局历史的分支预测方法中,也需要一个寄存器,来记录程序中所有的分支指令在过去的执行情况,这个寄存器被称为全局历史寄存器(Global History Register, GHR)。当然在现实当中,GHR寄存器不可能记录下所有分支指令的执行结果,因为GHR寄存器的宽度是不能无限大的。一般都是使用一个有限位宽的GHR,来记录最近执行的所有分支指令的结果,每当遇到一条分支指令时,就将这条分支指令的结果插入到GHR寄存器的右边,1表示发生了跳转(taken),0表示没有发生跳转(not taken),GHR寄存器最左边被移出的位会被抛弃掉。

在使用基于全局历史进行分支预测的方法时,每当要对一条分支指令进行分支预测,就会根据当前GHR寄存器的值来预测,此时仍然需要借助于PHT(也就是使用两位饱和计数器),使用GHR来寻址PHT,PHT仍然是由饱和计数器组成的,使用饱和计数器来捕捉GHR寄存器的规律。例如上面的例子,在对分支指令b3进行预测时,如果GHR寄存器的最低两位是11,即b3之前的分支b2和b1都执行了,那么b3就会预测不发生跳转(也就是预测值为0),最低两位是11的GHR值在PHT中对应的所有饱和计数器,经过一段训练

时间(training time)之后,都可以捕捉到这个规律,这些计数器就处于饱和状态了,之后就能进行正确分支预测。而如果在对分支指令 b3 进行预测时,GHR 最低两位的值并不是 11,那么 b3 就有可能发生跳转,也可能不跳转,这种并没有规律的情况,饱和计数器也无能为力,此时分支预测的准确度就不能保证了。

基于全局历史的分支预测方法,最理想的情况是对每条分支指令都使用一个 PHT,这样每条分支指令都会使用当前的 GHR 来寻址自身对应的 PHT,这种预测方法的示意图如图 4.20 所示。

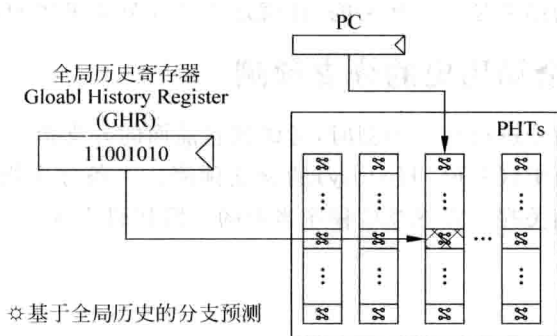


图 4.20 基于全局历史的分支预测算法中,对每条分支指令都使用一个 PHT

在图 4.20 所示的这种分支预测方法中,只使用一个 GHR 寄存器,用来记录最近执行的所有分支指令的结果,同时为每个 PC 都设置一个 PHT,所有的 PHT 组成的表格称为 PHTs。这样每条分支指令都可以在分支预测时,使用此时的 GHR 来寻址这条分支指令对应的 PHT,得到对应的分支预测结果,因为不同的分支指令有着不同的 PHT,所以即使两条分支指令对应的 GHR 相同,也不会寻址到同一个饱和计数器,因此也就不会产生冲突了。但是很显然,这种方法在实际当中是无法使用的,为每个 PC 值都使用一个 PHT 会占据非常大的存储空间,因此一般都会使用哈希(hash)法将 PC 进行处理,得到位宽很小的值,这样 PHTs 就可以包含少量的 PHT 了,如图 4.21 所示。

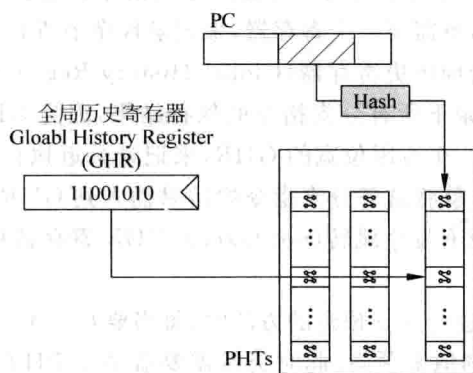


图 4.21 将 PC 值进行处理,从而减少 PHT 的个数

如图 4.21 所示的这种方法实际是 4.2.2 节中基于局部历史的分支预测方法的一种特殊情况,BHT 中只有一个 BHR 寄存器,此时这个 BHR 寄存器就对所有分支指令的执行结

果进行记录,也就是说,这个 BHR 寄存器成了本节所讲述的 GHR 寄存器,这就是由量变到质变的一个过程。当 BHT 减少到只有一个 BHR 寄存器时,就由基于局部历史的分支预测方法变为了基于全局历史的分支预测方法。

考虑到上面方法中,PHT 中有很多饱和和计数器都没有被使用,因此可以采用一种更简单的方式,即只使用一个 PHT,如图 4.22 所示。

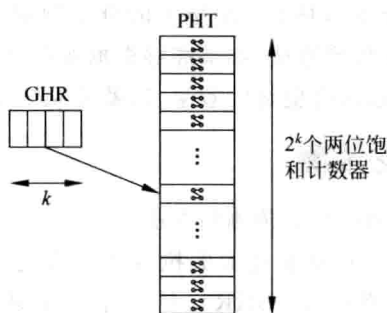


图 4.22 在基于全局历史的分支预测算法中,只使用一个 PHT

如图 4.22 所示的分支预测方法的缺点是如果两条不同的分支指令所对应的 GHR 值恰好相同的话,那么这两条分支指令就会共用 PHT 中的同一个饱和计数器,这样就造成了冲突。如果这两条分支指令朝着同一个分支方向(都发生跳转或者都不发生跳转),那么就对彼此的预测结果没有负面的影响;如果是朝着相反的分支方向,那么就会对彼此的预测结果造成负面影响了。

为了解决这个问题,仍旧可以考虑 4.2.2 节使用的方法,因为两条不同的分支指令的 PC 值是不一样的,可以将 PC 值和 GHR 做一定的处理,这个处理方法在基于局部历史的分支预测时已经使用过了,可以采用位拼接的方法,也可以采用异或(XOR)的方法,使用处理之后的结果来寻址 PHT,这样即使两条不同的分支指令对应同样的 GHR 的值,也不会对应到 PHT 中的同一个饱和计数器,避免了冲突的问题,如图 4.23 所示为这两种方法的示意图。

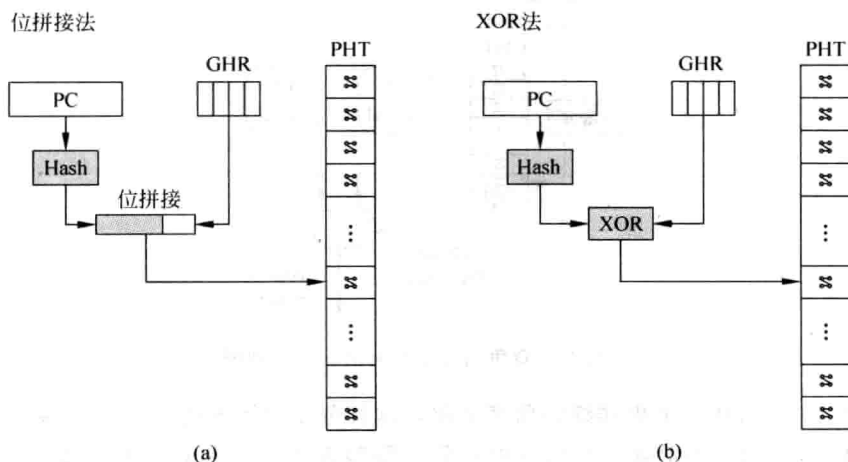


图 4.23 在基于全局历史的分支预测方法中使用位拼接法和异或法

从以前的例子可以看出来,使用异或(XOR)的方式可以避免很多冲突情况的发生,但是也会引入一些新的冲突,现实当中的处理器会使用一些更复杂的算法来避免冲突。基于全局历史的分支预测算法不能够保证对 4.2.2 节讲述的如 TNTNTN... 的分支指令进行完美的预测,因为此时可能受其他的分支指令结果的干扰,而且每次遇到这条分支指令,都需要重新对 PHT 进行训练(training),这样也降低了分支预测的准确度。造成这种现象的本质原因就是有些分支指令适合使用基于全局历史的分支预测方法,而另外一些分支指令则适合使用基于局部历史的分支预测方法,如果能够根据实际情况,对不同的分支指令使用不同的预测方法,那么这样就可以获得更优的效果,这就是 4.2.4 节要讲述的内容。

4.2.4 竞争的分支预测

到目前为止,主要介绍了两种分支预测的方法。

(1) 方法一: 基于分支指令自身在过去的执行状况来进行分支预测,这种方法对每条分支指令都使用了分支历史寄存器(BHR),并使用了由两位饱和计数器组成的 PHT (Pattern History Table)来捕捉每个 BHR 的规律,使用 BHR 和 PHT 配合来进行分支预测,这种方法称为基于局部历史(Local History)的分支预测法。

(2) 方法二: 基于一条分支指令之前的一些分支指令的执行状况来进行分支预测,使用了全局历史寄存器(GHR)来记录所有分支指令的执行情况,并使用了由两位饱和计数器组成的 PHT 来捕捉 GHR 的规律,使用 GHR 和 PHT 配合来进行分支预测,这种方法称为基于全局历史(Global History)的分支预测法。

这两种分支预测的方法各有优缺点,都有自身的局限性,有些分支指令使用基于局部历史的预测方法会有不错的效果,而有些分支指令则更适合使用基于全局历史的预测方法。可以设计一种自适应的分支预测方法,根据不同的分支指令的执行情况自动地选择这两种分支预测方法,Alpha 21264 处理器就使用了这样的预测方法^[47],称为竞争的分支预测(Tournament Predictor),就像是两种分支预测方法在进行竞争一样,如图 4.24 所示为这种分支预测方法的原理图。

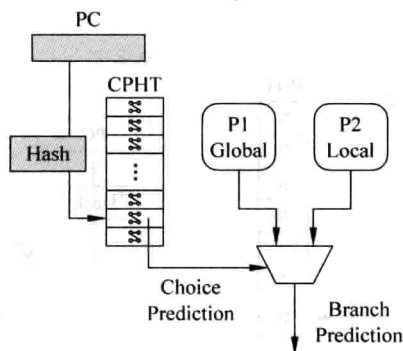


图 4.24 竞争的分支预测法——原理图

如图 4.25 所示从一个更详细的角度来解释这种分支预测方法的工作原理。

在图 4.25 中,P1 表示基于全局历史的分支预测方法,P2 表示基于局部历史的分支预测方法,对于不同的分支指令会选择不同的预测方法。CPHT(Choice PHT)是由分支指令

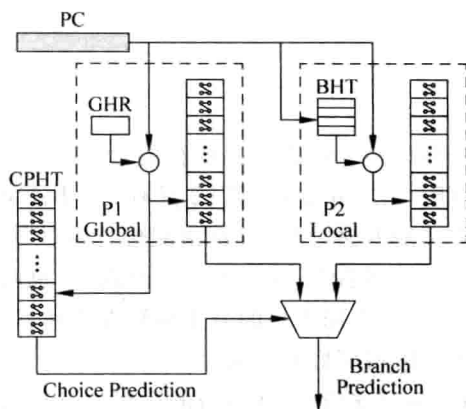


图 4.25 竞争的分支预测法——更详细的结构图

的 PC 值来寻址的一个表格,类似于 PHT,它仍是由两位的饱和计数器组成的,当其中一种分支预测方法(例如 P1)两次预测失败,而同时另外一种分支预测方法(例如 P2)两次预测成功时,会使状态机转到使用另一个分支预测方法的状态,CPHT 中每个饱和计数器的状态机如图 4.26 所示。

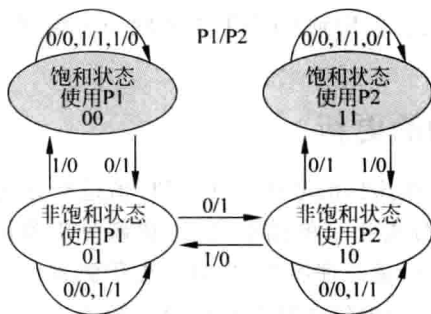


图 4.26 CPHT 中的两位饱和计数器

该状态机的转换机制如下。

- ① 当 P1 预测正确, P2 预测错误时(1/0), 计数器减 1;
- ② 当 P1 预测错误, P2 预测正确时(0/1), 计数器加 1;
- ③ 当 P1 和 P2 预测的结果一样时(1/1 或 0/0), 不管预测正确与否, 计数器都保持不变。

当计数器位于饱和的 00 状态时, 会使用 P1 进行分支预测; 位于饱和的 11 状态时, 会使用 P2 进行分支预测, 此时的预测准确度是比较高的。对于这种竞争的分支预测, 每条分支指令经过一段训练时间(training time), 都会根据两种分支预测方法的执行情况, 选择 P1 或者 P2 作为自身的分支预测的结果。但是, 这个选择并不是固定的, 也就是说, 对于一条特定的分支指令来说, 可能有时候适合使用基于全局历史的分支预测, 有时候适合使用基于局部历史的分支预测, 例如, 考虑下面的程序的执行情况。

```
if(aa == 0) / b1 /
    a = 0;
```

```

if(bb==0) / b2 /
    b = 1;
if(aa==bb)/ b3 /
    c = 3

```

当分支 b1 和 b2 都执行时,分支 b3 肯定是执行的,此时使用基于全局历史的分支预测方法对 b3 进行预测是合适的;而当分支 b1 和 b2 只有一个执行时,分支 b3 肯定不执行,此时使用基于全局历史的分支预测方法对 b3 进行预测也是合适的;而当分支 b1 和 b2 都不执行时,此时无法判断分支 b3 的条件(aa==bb)是否成立,也就无法确定分支 b3 是否执行,此时对分支 b3 来说,使用基于全局历史的分支预测方法并不是最合适的,有可能使用基于局部历史的分支预测方法能取得更好的效果。

由此可见,即使对于同一条分支指令,在执行的过程中,当 GHR 的内容不同时,可能会导致使用不同的分支预测方法,故将 PC 值和 GHR 寄存器进行一定的运算处理(例如进行异或),将运算的结果作为寻址 CPHT 的地址。同时从图 4.25 可以看出,该值也用来寻址 P1 分支预测方法中的 PHT,这样在进行分支预测时就将分支指令的地址和 GHR 寄存器都进行了考虑,可以对不同的分支指令在不同的执行情况下,采取最合适的分支预测方法。

从参考资料来看,Alpha 21264 处理器只使用了全局历史寄存器(GHR)作为寻址 CPHT 的地址,这样就相当于丢失了每条分支指令的地址信息,当两条不同的分支指令恰好对应的 GHR 值相同时,就会使用到 CPHT 中的同一个饱和计数器,这样就造成了冲突,降低了分支预测的准确度。

4.2.5 分支预测的更新

理想的分支预测方法遵循这样的顺序:根据 PC 值进行分支预测,使用预测的结果来继续执行,等到真正得到一条分支指令的结果之后,更新分支预测器(branch predictor)中的相关内容。但是在实际的超标量处理器当中,对分支预测器的更新需要考虑到流水线的影响,对于基于局部历史和全局历史的两种类型的分支预测方法,什么时候更新分支预测器,对分支预测的准确度是有影响的。对于分支指令的方向预测来说,需要更新的内容包括两个方面。

(1) 历史寄存器,在基于全局历史的分支预测方法中是 GHR,在基于局部历史的分支预测方法中是 BHR。

(2) 两位饱和计数器,不管是基于全局历史还是局部历史的分支预测方法,都需要在 PHT 中使用饱和计数器来捕捉历史寄存器的规律,因此也需要对饱和计数器进行更新。

1. 更新历史寄存器

在基于全局历史(Global History)进行分支预测的方法中,有三个时间点可以更新 GHR。

(1) 方法一:在流水线的取指令阶段,进行分支预测时,根据预测的结果更新 GHR;

(2) 方法二:当分支指令的方向被实际计算出来时,例如在流水线的执行阶段,更新 GHR;

(3) 方法三:当分支指令到达流水线最后的提交(Commit)阶段,要离开流水线而退休(retire)时,更新 GHR。

第三种方法显然是最保守的,当然最保守的方法也就意味着它是最安全的,当一条分支指令到达流水线的提交阶段,要离开流水线时(也称做这条分支指令要退休),该指令执行的所有状态都是确定的了,此时更新 GHR 肯定是万无一失的。但是考虑到现代的超标量处理器每周期要同时执行多条指令,并且有着很深的流水线,当一条分支指令退休时,该指令后面的很多条指令都已经进入流水线中,如果这些指令中存在分支指令,那么它们在进行分支预测时,都没有享受到这条分支指令的结果。仍旧考虑上一节的例子,当分支 b1 从流水线中退休时,分支 b2 和 b3 可能已经进入流水线中了,也就是说,分支 b3 在进行分支预测的时候,并没有使用到分支 b1 和 b2 的结果,因此就无法获得理论分析所能得到的预测准确度了。

更概括地说,一条分支指令 b 在时间 t 被分支预测,在时间 $t + \Delta t$ 从流水线中退休并更新 GHR,任何在 Δt 时间段之内被预测的分支指令,都不会从分支指令 b 的结果中获益(因为此时分支指令 b 的结果还没有写入到 GHR 当中)。尤其是当分支指令 b 前面存在产生 D-Cache 缺失的 load 指令时,会使分支指令 b 到达退休阶段所需要的时间变得更长,后续很多的分支指令在进行分支预测的时候,都无法使用分支指令 b 的结果,这就降低了分支预测的准确度,如图 4.27 所示给出了流水线处理器中,在分支指令退休的时候更新 GHR 所带来的问题。

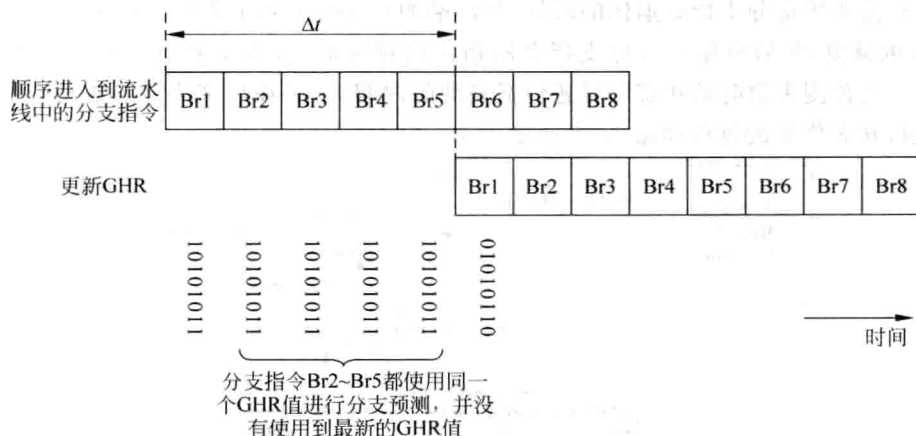


图 4.27 在流水线的 Commit 阶段更新 GHR 所遇到的问题

在图 4.27 中,分支指令 Br2~Br5 在进行分支预测的时候,都使用了同样的 GHR 值,这个 GHR 值在这段时间内并没有被更新,直到分支指令 Br1 退休的时候才会更新 GHR,在这种情况下,基于全局历史的分支预测方法形同虚设,分支预测的准确度也不会很高了。

使用方法二来更新 GHR 的时间要提前了很多,对于普通的处理器来说,流水线的执行阶段和取指令阶段的间隔只有几个周期(考虑经典的 IF、DC、EX、MEM 和 WB 流水线,EX 和 IF 之间只相差两个周期),所以在执行阶段更新 GHR,不会对后续的分支指令使用 GHR 进行分支预测产生太大的负面影响。而对于超标量处理器来说,流水线的执行阶段和取指令阶段之间存在的指令个数会很多,如果对分支指令采用了乱序执行的方式,在执行阶段的分支指令可能处于分支预测失败(mis-prediction)的错误路径中,即使是顺序执行分支指令的处理器,由于异常(exception)的处理需要清空流水线,分支指令在执行阶段的结果未必

一定会写到 GHR 中,因此在执行阶段更新 GHR 也并不一定是正确的。在计算机的术语中,这样的更新称为推测的(speculative),因为并不能保证此时 GHR 的更新是完全正确的,同时也会存在 GHR 无法及时更新的问题。

所以综合看起来,使用方法一,在取指令阶段,根据分支预测的结果对 GHR 进行更新是一个不错的做法,可以使后续的分支指令使用到最新的 GHR,而且,当一条分支指令的分支预测失败时,即使后续的分支指令都使用了错误的 GHR,其实也是没关系的,因为后续的这些分支指令都处在分支预测失败(mis-prediction)的路径上,都会被从流水线中抹掉,因此这些分支指令使用的 GHR 正确与否已经没有关系了。在取指令阶段更新 GHR 也是推测的(speculative),这种更新 GHR 的方式在分支预测失败时,需要一种机制对 GHR 进行修复,使 GHR 能够恢复到正确的值,下面介绍两种修复方法。

方法一:提交(Commit)阶段修复法。在流水线的提交阶段也放置一个 GHR,每当一条分支指令退休的时候,就会将它的结果更新到这个 GHR 中,这个 GHR 称做 Retired GHR,这样处理器中就有了两个 GHR:在前端取指令阶段的 GHR(称为 Speculative GHR),它用来进行分支预测,采用了推测(speculative)的方式进行更新;在后端提交阶段的 GHR,每当一条分支指令退休的时候才会更新它,因此这个 GHR 肯定是正确的。当一条分支指令发现分支预测失败的时候,表明此时前端的 GHR 肯定是错误的,需要进行修复,此时只需要等待分支指令退休的时候,将后端的 GHR 写到前端的 GHR 中,就完成了前端 GHR 的修复,然后根据这条分支指令所指定的目标地址,重新取指令来执行即可。当然,对于分支预测失败时的状态修复还包括其他的内容,这将在后文中逐步进行介绍,对于 GHR 进行状态修复的过程如图 4.28 所示。

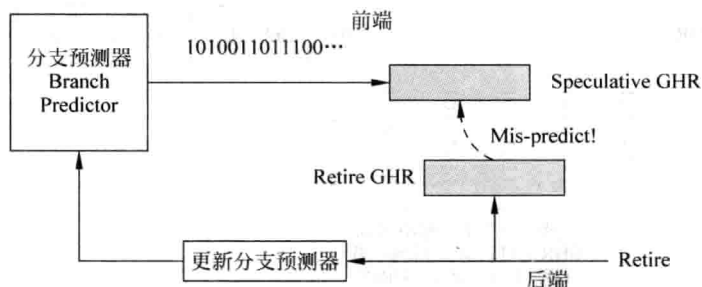


图 4.28 利用 Commit 阶段的 GHR,对分支预测器中使用的 GHR 进行恢复

使用图 4.28 所示的这种方法对 GHR 进行修复,缺点是会造成分支预测失败时惩罚(mis-prediction penalty)的增大,尤其是当分支指令前面有 D-Cache 缺失的 load 指令时,这种惩罚尤为明显,会严重降低处理器的性能。

方法二:Checkpoint 修复法。上面的方法在取指令阶段就可以对 GHR 进行更新,但是一旦发现分支预测失败时,只有等到分支指令退休的时候才可以对 GHR 进行状态修复,这中间间隔的时间过长,使流水线不能尽早地执行正确路径上的指令。其实,在取指令阶段对前端的 GHR 进行更新的同时,可以将旧的 GHR 值保存起来,这个保存的内容就称为 Checkpoint GHR。一旦这条分支指令的结果在流水线中被计算出来(例如在执行阶段),就可以对这条分支指令的分支预测是否正确进行检查了,如果发现分支预测正确,说明此时前端 GHR 中的值是正确的,那么继续执行就可以了;如果发现其分支预测失败,那么就将这

条分支指令对应的 Checkpoint GHR 恢复到前端的 GHR 中,并从这条分支指令正确的目标地址开始取指令来执行,这个过程如图 4.29 所示。

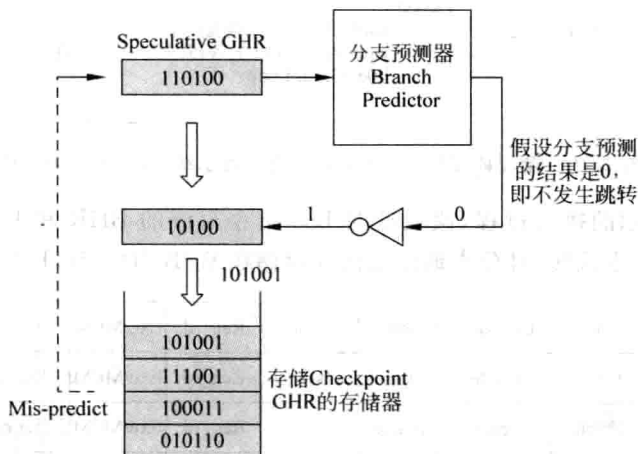


图 4.29 利用 Checkpoint 的方法对 GHR 进行恢复

在图 4.29 中,有一个存储器,专门用来保存所有 Checkpoint GHR 的值,由于新的分支指令的结果会从 GHR 的右边移入,此处将分支预测结果相反的值移入到 GHR,当发生分支预测失败时,就可以直接将这个值写到前端的 GHR 中,这样就完成了恢复。由于分支预测发生在取指令阶段,此时指令之间仍然保持着顺序(in-order)的方式,所以存储 Checkpoint GHR 的存储器只需要按照 FIFO 的方式进行写入就可以了;如果后续的流水线采用顺序的方式来执行分支指令,那么读取上述存储器也只需要按照 FIFO 的方式即可。但是,如果采用乱序(out-of-order)的方式来执行分支指令,那么读取这个存储器的顺序就不会按照顺序的方式了,这样增大了这个存储器的设计难度。

对于乱序执行分支指令的流水线来说,在执行阶段得到的分支指令的结果仍旧有可能是错误的,因为这条分支指令可能处于分支预测失败(mis-prediction)的路径上,也可能处于异常(exception)的路径上,所以仍旧需要在流水线的提交(Commit)阶段对分支指令的预测是否正确进行检查。正因为有这种需求,在提交阶段仍需要放置一个 Retired GHR,当分支指令退休的时候发现分支预测失败了,或者一条普通的指令发现了异常,都可以将此时的 Retired GHR 写回到前端的 GHR 中,这样就对 GHR 值进行了恢复。因此,从本质上来看,这种 Checkpoint 方法就相当于对方法一进行了补充,除了在流水线的提交阶段可以对 GHR 进行恢复,还可以在流水线的执行阶段对它进行恢复,这样就可以加快分支指令在分支预测失败时的修复时间,使处理器的执行效率得到提高。

而在基于局部历史的分支预测方法中,何时对 BHR 进行更新呢?这其实和基于全局历史的分支预测方法中遇到的情况是一样的,BHR 的更新可以是推测的(speculative),也可以是不推测的,不过和 GHR 有所不同,BHR 中存储的是当前的分支指令自身在过去的执行情况。一般情况下,只有循环体很短的情况,才可能出现一条分支指令在流水线的提交阶段更新 BHR 时,流水线中已经又出现了这条分支指令使用 BHR 进行分支预测的情况,例如图 4.30 所示的例子。

由于图 4.30 中的循环体只有三条指令(addi,addi,bne),在一个 3-way 的超标量处理器

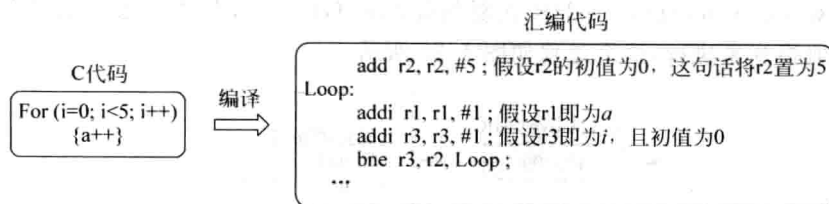


图 4.30 循环体很短时,分支指令进入流水线的间隔时间很短

中,有如图 4.31 所示的执行过程(假设此时 bne 指令对应的 BHR 和 PHT 都已经训练好了,即现在是预测发生跳转,且分支地址也已经存储在 BTB 当中,BTB 会在后文介绍)。

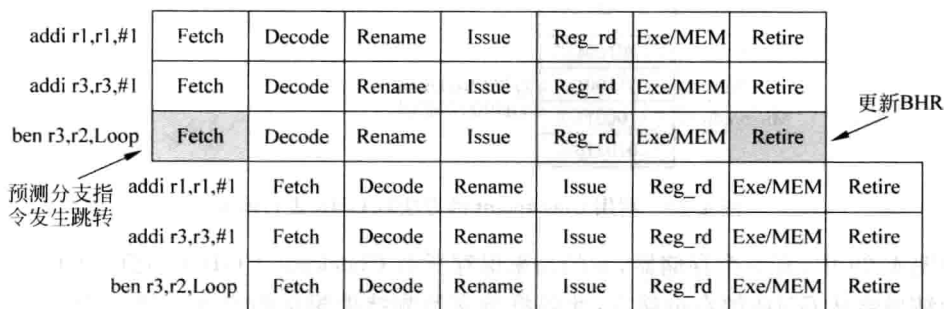


图 4.31 循环体很短时,流水线的执行情况

图 4.31 表示了流水线执行上述程序的情况,在第一个 bne 指令更新 BHR 之前,流水线中后续的所有 bne 指令都使用不到第一个 bne 指令的结果,但是这并不会对性能造成太大的影响,因为经过了一段训练时间(training time)之后,分支预测器会预测这个 bne 指令是发生跳转的,这样除了 for 循环的最后一次会发生预测失败之外,其他的时候都是预测正确的,这和基于全局历史的分支预测方法中对 GHR 的更新是有区别的。

总结起来就是,在基于全局历史的分支预测方法中,在取指令阶段根据分支预测的结果来更新 GHR 是最合适的,它利用了分支指令之间存在的相关性;而在基于局部历史的分支预测方法中,可以在分支指令退休的时候更新 BHR,这样可以简化设计,也不会对处理器的性能产生太大的负面影响。

2. 更新饱和计数器

不管是在基于全局历史的分支预测方法中,还是在基于局部历史的分支预测方法中,都需要对 PHT 中的饱和计数器进行更新,当一条分支指令比较有规律时,它对应的饱和计数器总是处于饱和状态,因此即使在分支指令退休的时候对 PHT 中的饱和计数器进行更新,也不会对分支预测的准确度产生很大的负面影响,所以在这两种分支预测方法中,一般都是在分支指令退休的时候对 PHT 中的饱和计数器进行更新。

4.3 分支指令的目标地址预测

在前面已经说过,分支指令的目标地址(target address)可以分为两种,即直接跳转(PC-relative)和间接跳转(absolute)。对于直接跳转的分支指令,由于它的偏移值(offset)

是以立即数的形式固定在指令当中,所以它的目标地址也是固定的,只要记录下这条分支指令的目标地址就可以了,当再次遇到这条分支指令时,如果方向预测的结果是发生跳转,那么它的目标地址就可以使用以前记录下的那个值;而对于间接跳转的分支指令来说,由于它的目标地址来自于通用寄存器,而通用寄存器的值是会经常变化的,所以对这种类型的分支指令来说,进行目标地址的预测并不是一件很容易的事情,但是庆幸的是,程序中大部分间接跳转类型的分支指令是用来处理子程序调用的 CALL 和 Return 指令,而这两种指令的目标地址是有规律可循的,因此可以对其进行预测。大部分超标量处理器都会推荐编译器的设计者,除了使用 CALL/Return 指令之外,尽量减少使用其他间接跳转类型的分支指令,而多使用直接跳转类型的分支指令,这样有助于处理器提高分支预测的准确度,从而提高处理器的性能。

4.3.1 直接跳转类型的分支预测

1. BTB

对于直接跳转(PC-relative)类型的分支指令来说,它的目标地址有两种情况。

(1) 当分支指令不发生跳转时,

目标地址 = 当前分支指令的 PC 值 + Sizeof(fetch group)。

(2) 当分支指令发生跳转时,

目标地址 = 当前分支指令的 PC 值 + Sign_extend(offset)。

分支预测器(branch predictor)除了需要对分支指令的方向进行预测之外,还需要对目标地址也进行预测。对于 RISC 指令集来说,由于计算地址所需要的偏移值(offset)是在指令中以立即数的形式进行的携带,所以这个偏移值是不会发生变化的,因此对于一条特定的直接跳转类型的分支指令来说,由上面的表达式可以得知,它的目标地址是不会随着程序的执行而变化的(此处不考虑代码自修改的情况,因为执行完这种代码之后,需要对分支预测器进行复位),因此对这种分支指令的目标地址进行预测是很容易的,只需要使用一个表格,记录下每条直接跳转类型的分支指令对应的目标地址,当再次对这条分支指令进行预测时,只需要查找这个表格就可以得到预测的目标地址了。

由于分支预测是基于 PC 值进行的,不可能对每一个 PC 值都记录下它的目标地址,所以一般都使用 Cache 的形式,使多个 PC 值共用一个空间来存储目标地址,这个 Cache 称为 BTB(Branch Target Buffer),图 4.32 表示了采用直接映射结构的 BTB 的示意图。

BTB 本质上是 Cache,所以它的结构和 Cache 也是一样的,使用 PC 值的一部分来寻址 BTB(这部分称为 index),PC 值的其他部分作为 Tag。BTB 中存放着分支指令的目标地址(Branch Target Address,BTA),因为 Index 部分相同的多个 PC 值会查找到 BTB 中的同一个地方,所以使用 Tag 来进行区分,当这些 PC 值中存在多于一条的分支指令时,就产生了冲突,这样会造成 BTB 中对应的内容被频繁地替换,影响了分支预测的准确度。

为了解决这个问题,可以采用组相连结构的 BTB,当有多条分支指令都指向 BTB 的同一个地方时(也就是它们的 PC 值的 Index 部分相同),可以将它们的 BTA 放到不同的 way 中,如图 4.33 所示。

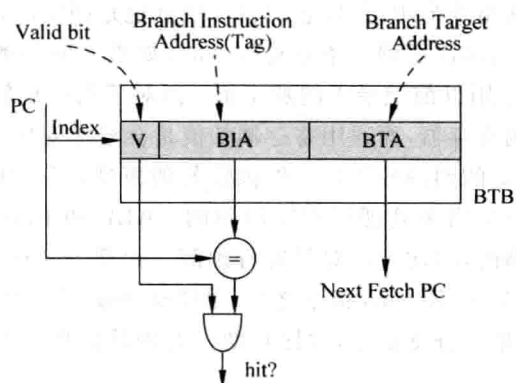


图 4.32 Direct-mapped BTB

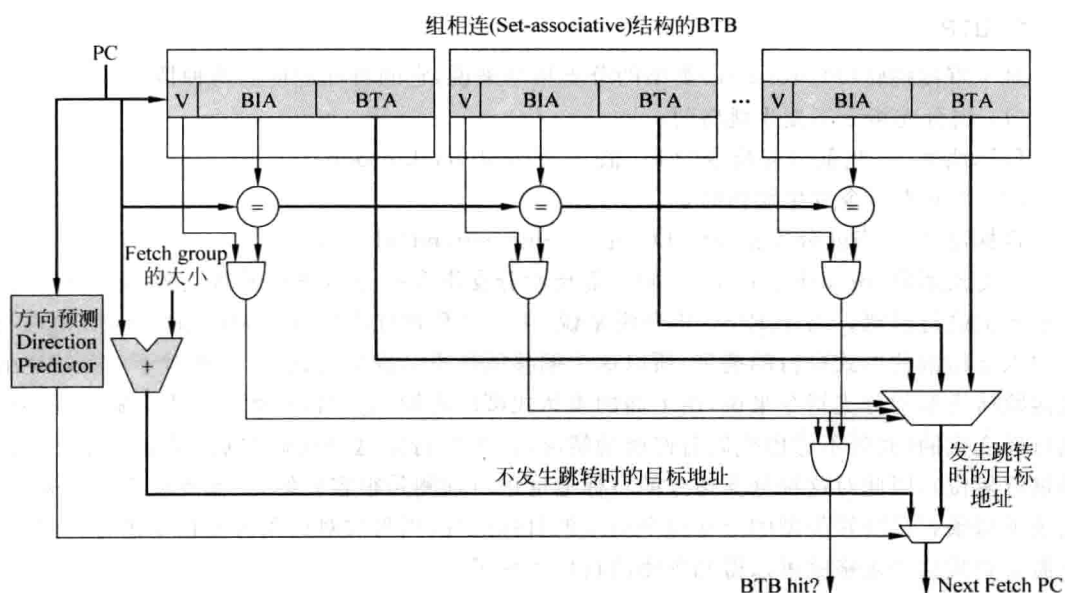


图 4.33 Set-associative BTB

当然，组相联结构的 BTB 增加了设计的复杂度，使 BTB 占用的硅片面积增大，同时降低了 BTB 的访问速度，因此在现实世界的处理器中，BTB 中 way 的个数一般都比较小。

以上所讲述的 BTB 的设计都是比较直接的方法，还可以采用一些其他的方法来对其进行优化，例如在直接映射结构的 BTB 中，最优的情况出现在映射到 BTB 中同一个地方的所有指令中，最多只有一条是分支指令，在这种最优的情况下，可以将 BTB 的 Tag 部分进行简化，只使用 PC 值的一小部分作为 Tag，如图 4.34 所示。

图 4.34 所示的这种方法称为 partial-tag BTB，它可以节省 Tag 所占据的存储空间，但是当 BTB 中的一个地方对应多条分支指令时，这种方法就有可能引起目标地址的预测失败，例如图 4.34 所示的 PC 值 (beef9810) 也对应着一条分支指令，那么就会错误地使用之前的 PC 值 (cfff9810) 对应的目标地址，造成目标地址预测的失败，从这一点来看，似乎这种方法并不是一个很好的选择，但是需要注意的是，即使不采用这种优化的方法，在普通的直接

映射结构 BTB 中,这种情况也会由于冲突而引起分支指令(beef9810)的目的地址预测的失败,所以从上面的这个例子来看,在直接映射结构的 BTB 中使用图 4.34 所示的这种减少 Tag 位数的方法,并不会降低目标地址预测的正确率。当然,如果 PC 值(beef9810)对应的指令并不是一条分支指令,此时也不会有任何问题,因为方向预测会得到这条指令是不发生跳转的,也就不会使用这个错误的目标地址了。因此综合来看,在直接映射结构的 BTB 中使用图 4.34 的这种减少 Tag 位数的方法,并不会引起分支预测正确率的降低,还可以节省 BTB 的存储空间,是一种行之有效的改进方法。

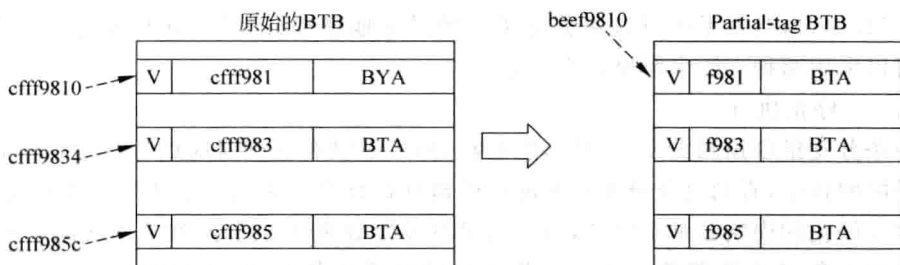


图 4.34 缩短 BTB 中 Tag 的位数,从而降低 BTB 的面积

还需要注意的是,分支预测失败并不会造成处理器运行的错误,因为在流水线的后续阶段还会对分支预测的正确与否进行检查,如果发现一条分支指令的分支预测不正确,那么会将这条分支指令之后进入到流水线的指令从流水线中抹掉,并从正确的地方开始取指令,当然,此时处理器做了一部分无用功,会浪费一些性能和功耗。

当然,partial-tag BTB 的这种直接将原来的 Tag 砍掉了一部分,这相当于丢失了原来的一部分信息,其实还可以采用之前使用过的方法,将原来的 Tag 进行一定的运算,从而得到一个位数比较小的值。举例来说,对于图 4.34 中 28 位的 Tag,对其每相邻的两个四位的值进行异或运算,这样就可以得到一个 14 位的新值作为 Tag,如图 4.35 所示。

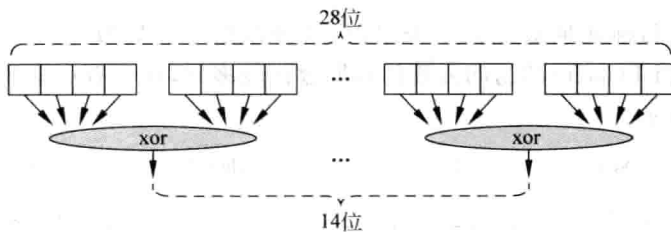


图 4.35 采用异或的方法降低 Tag 的位数

通过这种方式,可以减少 Tag 的位数,同时可以避免不同的 PC 值之间重名的情况,可以计算出,两个 PC 值 beef9810 和 cff9810 经过上述的运算处理之后的值是不相同的。这种对 Tag 进行一定的运算处理的方法应用于组相连接结构的 BTB 中,可以在面积较小的情况下,获得不错的性能。

一般情况下,为了最大限度地利用 BTB 中有限的存储资源,只将发生跳转的分支指令对应的目标地址放到 BTB 中,那些预测不发生跳转的分支指令,它们的目标地址其实就是顺序取指令的地址,因此并不需要进行预测。从本节的描述可以看出,对于直接跳转(PC-

relative)类型的分支指令来说,使用 BTB 进行目标地址的预测是比较准确的,但是对于间接跳转(absolute)类型的分支指令,由于它的目标地址是经常变化的,所以使用 BTB 对目标地址进行预测,是无法得到满意结果的,需要采用一些其他的方式来解决,这在后文中会进行介绍。

2. BTB 缺失的处理

当一条分支指令的方向是预测发生跳转,而此时 BTB 发生了缺失,例如在直接映射结构的 BTB 中,多条分支指令对应着同一个表项,那么此时这条分支指令的目标地址就有可能不在 BTB 中,当然就无法对这条分支指令的目标地址进行预测,这时候应该怎么办呢?处理器可以采用两种方法来解决这个问题。

方法一: 停止执行

当一条分支指令预测会发生跳转,但是在 BTB 中没有找到对应的目标地址时,可以暂停流水线的取指令,直到这条分支指令的目标地址被计算出来为止。不同类型的分支指令导致流水线停止的周期数是不同的,对于直接跳转类型的分支指令,在流水线的解码阶段就可以从指令中分离出偏移值(offset),此时就可以将这条分支指令的目标地址计算出来($PC + offset$),这个过程示意图如图 4.36 所示。

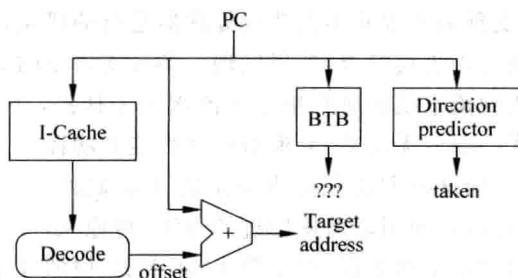


图 4.36 在流水线的解码阶段,计算得到分支指令的目标地址

在分支指令的目标地址被计算出来之前,暂停取指令会导致流水线中出现一些气泡(Bubble),尤其是当 I-Cache 的访问需要的周期数比较多时,就会有更多的气泡出现在流水线中,如图 4.37 所示。

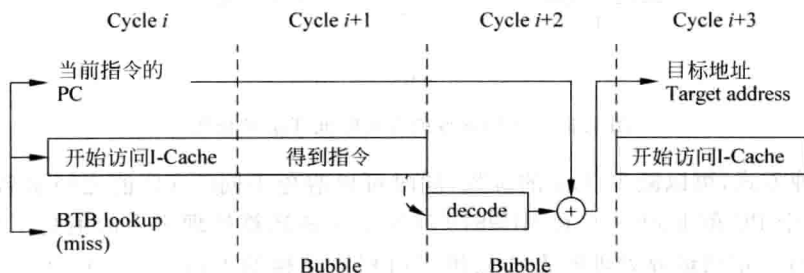


图 4.37 BTB miss 时,暂停流水线会引入气泡

图 4.37 所示中,在周期 i 的时候发现了预测跳转的分支指令,但是却没有在 BTB 中发现这条指令的目的地址,这时候就要停止取指令,并等待分支指令从 I-Cache 中取出来。在图 4.37 中表示的 I-Cache 需要两个周期才可以得到指令,因此在周期 $i+2$ 就可以对分支指

令进行解码而得到指令中的偏移值,进而就可以计算出分支指令的目标地址了,在周期 $i+3$ 可以使用这个计算出的地址进行取指令,通过图 4.37 所示可以看出,这种停顿造成流水线中引入了两个周期的气泡。

使用这种停止取指令的方法处理 BTB 的缺失,对于直接跳转(PC-relative)类型的分支指令是比较有效率的,因为这种类型的分支指令的目标地址可以很快地计算出来,但是对于间接跳转(absolute)类型的分支指令,它的目标地址需要等到通用寄存器的值被计算出来,这个过程的时间可能会很长(例如这个通用寄存器来自于发生 D-Cache 缺失的 load 指令),这会导致流水线中产生大量的气泡,降低了处理器的执行效率。

方法二:继续执行

对于方向预测会发生跳转的分支指令来说,如果发生了 BTB 的缺失,此时可以使流水线继续使用顺序的 PC 值来取指令。当在流水线的后续阶段,将它的目标地址计算出来之后,如果发现计算出的地址和原来顺序的 PC 值不一样(这个可能性是非常高的),那么就将流水线中分支指令之后进入到流水线的指令都抹掉,使用计算出的目标地址开始取指令。之所以这样做,是考虑到分支指令的方向可能会预测错误,也就是分支指令的实际结果可能是不发生跳转的,所以对于这些预测发生跳转的分支指令,当 BTB 发生缺失时,采用顺序的 PC 值来取指令,也存在正确的可能性。尤其是对于间接跳转(absolute)类型的分支指令,要等到它的目标地址被真正地计算出来,需要的时间可能会很长,顺序执行总比暂停流水线要更“聪明”一些,因为毕竟存在正确的可能性,“better later than never”。但是从功耗的观点来看,这样做是更浪费功耗的,因此采用这种方法可能需要经常从流水线中抹掉指令,这等于是在做无用功,所以如果是对功耗比较敏感的设计,这种方法可能并不是一个好的选择。

4.3.2 间接跳转类型的分支预测

对于间接跳转(absolute)类型的分支指令来说,它的目标地址来自于通用寄存器,是经常变化的,所以无法通过 BTB 对它的目标地址进行准确的预测,所幸的是,大部分间接跳转类型的分支指令都是用来进行子程序调用的 CALL/Return 指令,而这两种指令是有规律可循的,任何有规律的事情都可以进行预测。对于除了 CALL/Return 类型之外的其他间接跳转类型的分支指令,很多人也对其分支预测进行了研究,并提出了一些方法来提高其预测的准确度,本节将会进行介绍。

1. CALL/Return 指令的分支预测

在一般的程序中,CALL 指令用来调用子程序,使流水线从子程序中开始取指令执行,而在子程序中,Return 指令一般是最后一条指令,它将使流水线从子程序中退出,返回到主程序中的 CALL 指令之后,继续执行。对于很多 RISC 处理器来说,可能在指令集中并没有直接的 CALL/Return 指令,而是使用其他的指令来模拟这个功能,例如在 MIPS 处理器中,使用 JAL 指令作为 CALL 指令,而使用“JR \$31”指令作为 Return 指令。对于程序中的一条指定的 CALL 指令来说,它每次调用的子程序都是固定的,也就是说,一条 CALL 指令对应的目标地址是固定的,因此可以使用 BTB 对 CALL 指令的目标地址进行预测,如图 4.38 所示给出了一个例子。

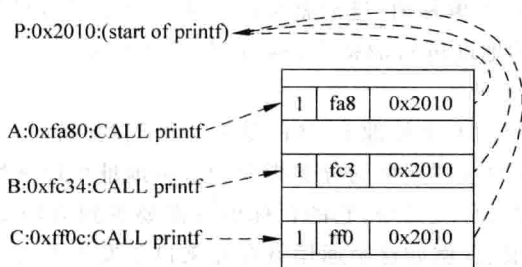


图 4.38 CALL 指令使用的目标地址都是固定的,可以使用 BTB 进行目标地址的预测

如图 4.38 所示中,printf 函数的起始地址是 0x2010,三条不同的 CALL 指令都调用了 printf 函数,因为这三条指令的 PC 值不同,所以它们占据了 BTB 中的不同地方,由于被调用的 printf 函数的地址是固定的,所以这三条 CALL 指令都可以使用 BTB 对目标地址进行正确的预测。

但是对于一个子函数来说,例如 printf 函数,有很多地方都可以调用它,因此 printf 函数执行到最后一条 Return 指令的时候,返回的地方是会变化的。例如图 4.38 中,CALL 指令 A 调用了 printf 函数,则 printf 函数中的 Return 指令会返回到指令 A 之后的那条指令;同理,CALL 指令 B 调用了 printf 函数的话,printf 函数中的 Return 指令就会返回到指令 B 之后的那条指令。正因为 Return 指令的目标地址是不确定的,因此无法使用 BTB 对它的目标地址进行预测,但是可以看出,Return 指令的目标地址总是等于最近一次执行的 CALL 指令的下一条指令的地址,如图 4.39 所示。

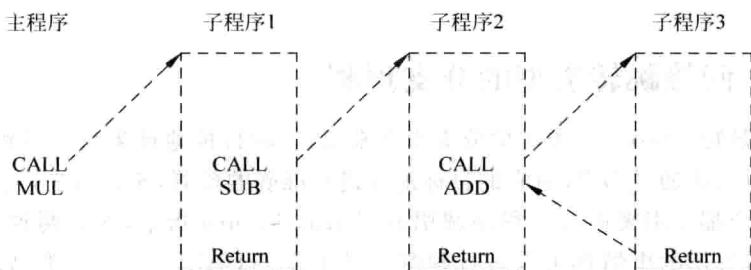


图 4.39 一个三级嵌套的子程序调用

如图 4.39 所示为一个三级嵌套的子程序调用,每个子程序中都调用了其他的子程序,当执行到子程序 3 的 Return 指令时,它的目标地址是最近一次执行的 CALL 指令(位于子程序 2 中)的下一条指令(ADD 指令)的地址;同理,当执行子程序 2 的 Return 指令时,它的目标地址是子程序 1 的 CALL 指令的下一条指令(SUB 指令)的地址,也就是说,Return 指令的目标地址,是按照 CALL 指令执行的相反顺序排列的。

根据 Return 指令的上述特点,可以设计一个存储器,保存最近执行的 CALL 指令的下一条指令的地址,这个存储器是后进先出的(Last In First Out, LIFO),即最后一次进入的数据将最先被使用,这符合上面讲述的 Return 指令和 CALL 指令的特点,这个存储器的工作原理和堆栈(stack)是一样的,称之为返回地址堆栈(Return Address Stack, RAS)。对于图 4.39 所示的例子,当执行完三条 CALL 指令之后,RAS 如图 4.40 所示。

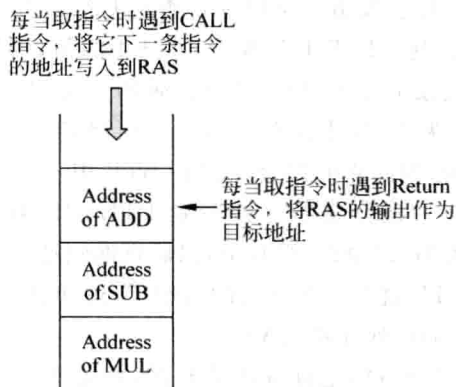


图 4.40 执行三条 CALL 指令之后, RAS 中的值

当对子程序 3 中的 Return 指令进行目标地址的预测时,会使用 RAS 最新进入的地址,也就是 ADD 指令的地址,这样就能够返回到子程序 2 中继续执行,同时 RAS 的读指针会指向下一个值;当对子程序 2 的 Return 指令进行目标地址的预测时,会使用此时 RAS 中的最新地址,也就是 SUB 指令的地址作为 Return 指令的目标地址,这样可以返回到子程序 1 中继续执行,同时 RAS 的读指针会指向下一个值;最后对子程序 1 的 Return 指令进行目标地址预测时,就会使用 RAS 中的 MUL 指令的地址作为目标地址了,这样就可以返回到主程序中继续执行。因此综合看起来,可以使用 BTB 对 CALL 指令的目标地址进行预测,而使用 RAS 对 Return 指令的目标地址进行预测,这个过程如图 4.41 所示。

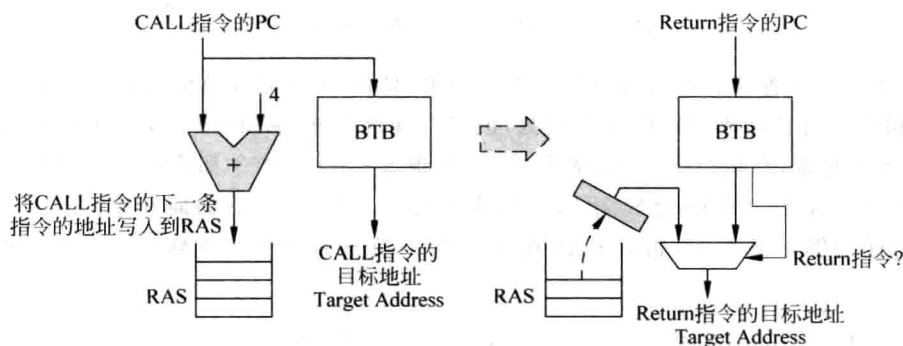


图 4.41 对 CALL/Return 指令进行分支预测

由图 4.41 可以看出,要使 RAS 能够正确工作,需要如下两个前提条件。

(1) 当遇到 CALL 指令的时候,需要能够将 CALL 指令的下一条指令的地址放到 RAS 中,这需要识别出哪条指令是 CALL 指令,而正常情况下,只有到了流水线的解码阶段才可以知道指令是否是 CALL 指令,此时可以将 CALL 指令的 PC+4 值保存到 RAS 中(假设 PC 值会和指令一起随着流水线而流动),但是现代的处理器都需要几个周期才可以从 I-Cache 中取出指令,因此当指令到达流水线的解码阶段的时候,在这条指令后面已经有很多条指令又进入了流水线中,如果在这些指令中存在 Return 指令(假设子函数很短),那么这条 Return 指令将无法从 RAS 中得到正确的目标地址,这样就造成了分支预测失败,降低了处理器的执行效率。

如果能够在分支预测阶段就知道一条指令是否是 CALL 指令(即通过 PC 值就可以知道是否是 CALL 指令),那么就可以及时地保存 PC+4 的值,即使 CALL 指令后面马上跟着 Return 指令,也可以对这条 Return 指令进行正确的分支预测,这个功能要如何实现呢?这仍旧需要借助于 BTB,因为 BTB 中保存了所有发生跳转的分支指令,而 CALL/Return 指令都是永远会发生跳转的,因此它们都会保存在 BTB 中,这需要在 BTB 中增加一项,用来标记分支指令的类型,当一条分支指令被写到 BTB 的时候,也会将分支指令的类型(CALL、Return 或者其他类型)记录在 BTB 中,以后再遇到这条分支指令时(也就是再遇到这个 PC 值时),通过查询 BTB 就可以知道它的类型了,这样就可以在分支预测阶段识别出 CALL 指令,从而将 PC+4 的值保存在 RAS 中。

(2) 当对 Return 指令进行目标地址预测的时候,需要能够选择 RAS 的输出作为目标地址的值,而不是选择 BTB 的输出值,因此仍需要在分支预测阶段就可以知道指令的类型,这依然可以通过如上所述的方法,在 BTB 中存储分支指令的类型,这样在分支预测阶段访问 BTB 的同时,就可以知道当前的指令是否是 Return 指令了。

因此,在 BTB 中,除了存储目标地址和 Tag 值之外,还需要存储分支指令的类型,图 4.42 表示了这样的 BTB。

2bit			
V	BIA(tag)	BTA	Br_type

BTB

图 4.42 将指令的类型存储在 BTB 中

到目前为止,通过上述的理论分析可以知道,使用 BTB 和 RAS,可以对 CALL 指令和 Return 指令进行正确率很高的分支预测,但是在实际当中,如果一个程序中 CALL 指令嵌套很深,也就是说,在执行 Return 指令之前,有很多 CALL 指令都要将下一条指令的地址放到 RAS 中,那么当 RAS 已经满了时,如果再有 CALL 指令,该如何处理?

以这样的例子来考虑,如果 RAS 的大小为 4,而子程序嵌套的级数为 5 时,如图 4.43 所示。



图 4.43 在 RAS 的大小为 4 的处理器中执行 5 级嵌套的子程序调用

在如图 4.43 所示的例子中,子程序 3 中的 CALL 指令被分支预测后,RAS 就已经被占满了,当子程序 4 中的 CALL 指令要进行分支预测时,RAS 中已经没有空余的地方来存储它的返回地址,如图 4.44 所示。

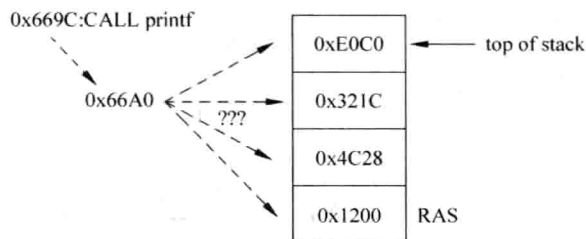


图 4.44 RAS 中已经没有空间来存储返回地址

有两种方法可以处理上述情况。

(1) 不对新的 CALL 指令进行处理,此时不修改 RAS,最后执行的这个 CALL 指令对应的返回地址(0x66A0)将会被抛弃掉,这样在下一次执行 Return 指令的时候肯定就会产生分支预测失败(mis-prediction),不仅如此,这种做法还要求 RAS 的指针不能够发生变化,否则就会引起后面的 Return 指令都无法对应到自己的返回地址,显然,这是一个比较差的做法。

(2) 继续按照顺序向 RAS 中写入,此时 RAS 中最旧的那个内容(存放着地址 0x1200)将会被覆盖掉,如图 4.45 所示。

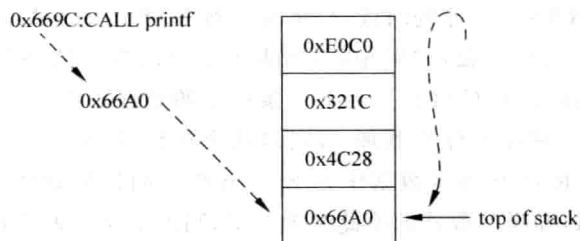


图 4.45 继续向 RAS 中写入,覆盖掉最旧的值

按照这样的处理方式,对于子程序 5、4、3、2 中的 Return 指令都可以正确地预测其目标地址,但是对于子程序 1 中的 Return 指令,因为将它对应的返回地址进行了覆盖,此时将无法得到正确的目标地址,因此会引起不可避免分支预测失败。

但是比较来说,这种方法要更好一些,因为它存在正确的可能性,考虑一个递归的(recursive)函数调用: f()调用 f(),即自己调用自己,如图 4.46 所示。

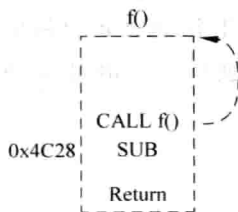


图 4.46 递归函数

如果 RAS 的容量仍旧只有 4 个,则在这个递归函数第 5 次调用的时候,RAS 的情况如图 4.47 所示。

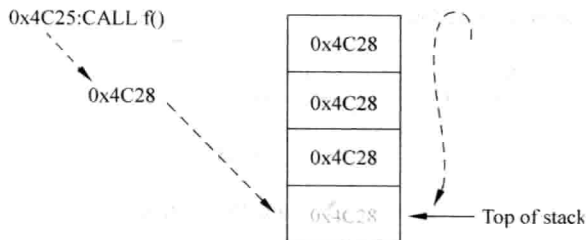


图 4.47 递归调用时,写入到 RAS 的值和被覆盖的值是一样的

由图 4.47 所示可以看出,RAS 中被覆盖的目标地址和写入的地址是一样的,这样即使进行了覆盖,RAS 仍能够给出正确的返回地址,这也是使用方法(2)的一个优点。

不过,像图 4.47 中这种递归的函数调用,RAS 都被同一个 CALL 指令的返回地址所占据了,RAS 中保存的都是同一个重复的值,这样对 RAS 空间来说,其实是浪费的。对于连续执行的同一个 CALL 指令来说(此处的连续是指,两次相邻执行的 CALL 指令是同一条指令),完全可以将它们的返回地址都放到 RAS 中的同一个地方,并用一个计数器来标记 CALL 指令执行的次数,例如使用一个 8 位的计数器就可以最多标记 256 级的递归调用了,这样相当于扩展了 RAS 的容量,增大了预测的准确度。

这种带计数器的 RAS 在工作的时候,会将写入到 RAS 中的 CALL 指令的返回地址和上一次写入的返回地址进行比较,如果相等,则表示这两次执行的都是同一个 CALL 指令(RAS 中保存的返回地址就是 CALL 指令下一条指令的地址),则此时要将 RAS 当前的读指针指向的表项(entry)对应的计数器加 1,同时这个指针保持不变;当遇到 Return 指令时,按照以前的方式从 RAS 中读取数据作为 Return 指令的目标地址,同时将 RAS 当前的这个表项的计数器减 1,如果计数器的值此时为 0 了,则表示这个表项对应的 CALL 指令已经结束了递归调用,此时可以使 RAS 的读指针指向下一个 CALL 指令的返回地址,通过这种工作方式,可以很好地对递归函数进行分支预测。

2. 其他预测方法

对于间接跳转(absolute)类型的分支指令,如果它既不是 CALL 指令,又不是 Return 指令时,该如何预测它的目标地址呢?从理论上讲,这种类型的分支指令,它的目标地址可能有很多,对于 32 位的处理器来说,这个目标地址有 2^{30} 种可能性,是没有办法进行预测的。但是这只是理论上的情况,在实际的程序中,对于一个间接跳转(absolute)类型的分支指令来说,它的目标地址只有固定的几个,因此可以根据分支指令在过去的执行情况,对目标地址进行预测,一个很明显的例子就是 case 语句,如下所示:

```
case(a)
  1: 跳转到目标地址 1
  2: 跳转到目标地址 2
  3: 跳转到目标地址 3
  ⋮
  9: 跳转到目标地址 9
```

如果这个 case 语句被编译器翻译成了间接跳转类型的分支指令,但是因为 case 语句的内容有限,分支指令的目标地址也是有限的几个,这就为目标地址的预测提供了可能性,很

多人对这种类型的分支指令如何预测进行了研究,本节给出一种方法^[17]。

对于间接跳转类型的分支指令来说,它的目标地址也可能是和过去的执行情况有关系的,因此可以利用基于局部历史的分支预测方法中使用的 BHR 对目标地址进行预测,如图 4.48 所示。

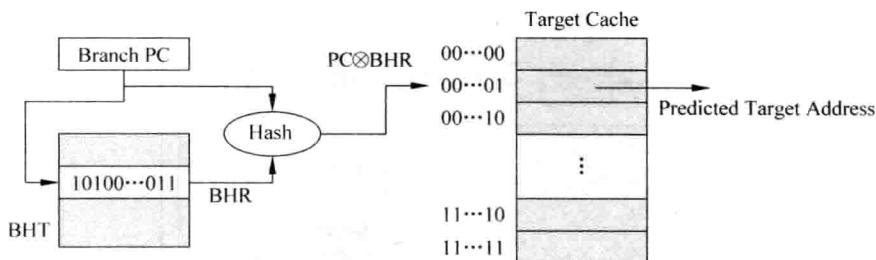


图 4.48 使用基于局部历史的分支预测方法对目标地址进行预测

如图 4.48 所示的方法已经在分支指令的方向预测中出现过,只是现在将 PHT 换成了 Target Cache。对于一条间接跳转类型的分支指令来说,每当它执行一次时,就按照图 4.48 所示的方式将目标地址写到 Target Cache 中。上面说过,一条间接跳转类型的分支指令,它的目标地址可能出现的值是有限的,所以使用 Target Cache 有可能覆盖到所有的情况。

因此,到目前为止,对于分支指令的目标地址的预测,有下面的三种方法。

- (1) 使用 BTB 对直接跳转(PC-relative)类型的分支指令和 CALL 指令进行预测;
- (2) 使用 RAS 对 Return 指令进行预测;
- (3) 使用 Target Cache 对其他类型的分支指令进行预测。

尤其是对于 BTB 和 RAS,几乎是现代的超标量处理器必须要使用的。

4.3.3 小结

到目前为止,对于分支指令的方向和目标地址的预测方法都进行了讲述:使用 BHR、GHR 和饱和计数器配合来对分支指令的方向进行预测,并使用 BTB、RAS 和 Target Cache 对分支指令的目标地址进行预测,这些预测都是发生在取指令的阶段,基于 PC 值来进行的。将以上的方法进行汇总,就可以对分支指令进行完整的分支预测了,图 4.49 给出了一种典型的实现方法。

图 4.49 中使用了基于局部历史的方法对分支指令的方向进行预测,配合使用 BTB、RAS 和 Target Cache 对分支指令的目标地址进行预测,这种将分支指令的方向预测独立于 BTB 的做法称为 decoupled BTB,这样那些本来没有在 BTB 中记录的分支指令也会被进行分支方向的预测,由于 BTB 需要占用不菲的硬件资源,一般它的容量都不会很大,也就是无法记录很多条的指令信息,但是进行分支方向预测的 BHT 和 PHT 则可以记录很多条指令的信息,这样可以对更多的分支指令进行方向的预测。即使某些分支指令被预测为发生跳转,但是却发生了 BTB 缺失,仍可以通过将处理器暂停取指令来节约功耗,这也比发生分支预测失败的情况要好一些。举例来说,在 PowerPC 604 处理器中使用了一个 64-entry,全相连接结构的 BTB,用来保存最近发生跳转的分支指令的目标地址,同时使用了一个 512-entry 的 PHT 来进行分支方向的预测^[18],这就是典型的 decoupled BTB 的设计。

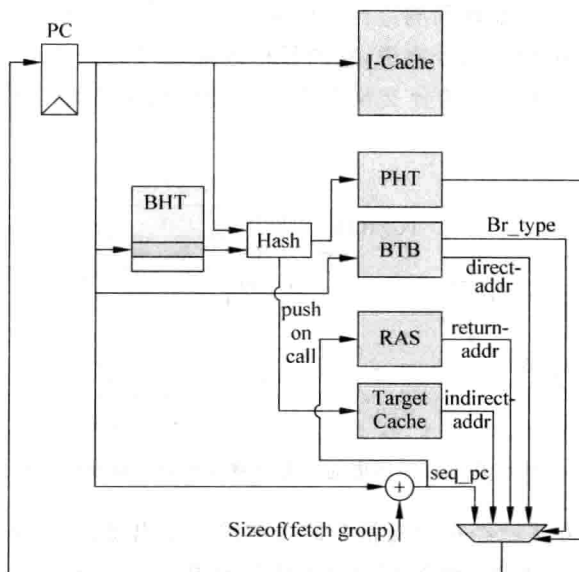


图 4.49 一种完整的分支预测方法

当然,任何预测技术都有可能出错,分支预测也不例外,因此就需要一套机制对分支预测的正确性进行检查,在超标量处理器的流水线中,很多阶段都可以得到分支指令的结果,例如最早在解码阶段就可以得到直接跳转类型(PC-relative)的分支指令的结果,而在执行阶段可以得到间接跳转类型(absolute)的分支指令的结果等。在流水线中得到一条分支指令的结果时,就可以对这条分支指令的分支预测是否正确进行检查,发现分支预测失败时,在这条分支指令后面进入到流水线的所有指令都处在了错误(mis-prediction)的路径上,这些指令都应该从流水线中被抹掉,这就造成了流水线中存在很多的气泡(bubble),降低了处理器的性能,这称为分支预测失败时的惩罚(mis-prediction penalty)。这些处在错误路径上的指令有可能已经将处理器中某些部件的内容进行了更改,例如寄存器重命名阶段的重命名映射表(mapping table),这个更改显然是不应该的,需要对这些操作进行撤销,这称为分支预测失败时的恢复(mis-prediction recovery),在 4.4 节将对这些内容进行介绍。

4.4 分支预测失败时的恢复

分支预测是一种预测技术,需要处理器中有对应的验证和恢复机制,在流水线的很多阶段都可以对分支预测是否正确进行检查,它们包括以下几点。

(1) 在解码阶段,可以得到一个 PC 值对应的指令是否是分支指令,以及这条分支指令的类型,如果这条指令是直接跳转(PC-relative)类型的分支指令,还可以在这个阶段得到它的目标地址,因此在解码阶段可以对一部分的分支指令是否预测正确进行检查,例如在这个阶段发现了一条直接跳转类型的无条件分支指令(例如 MIPS 中的 J 指令),则这条分支指令的方向(发生跳转)和目标地址(PC+offset)就可以得到了。如果发现这条分支指令在之前被预测为不发生跳转,则此时就发现了分支预测失败(mis-prediction),这条本来应该发生跳转的分支指令并没有起作用。不过对于分支预测失败来说,如果在解码阶段就发现它,

那么是最好的事情了,因为此时可以马上进行恢复,这样所带来的惩罚(penalty)也是最小的。当然,很多时候即使在解码阶段发现方向预测不正确,可能也无法在这个阶段得到正确的目标地址,例如一条目标地址位于寄存器中的间接跳转指令(例如 MIPS 中的 JR 指令),如果它在之前被预测为不发生跳转,则此时在解码阶段就可以发现这个错误,但是在这个阶段却无法知道寄存器的内容,因为读取寄存器的过程发生在流水线的后续阶段,因此即使知道这条指令预测失败了,此时也没有办法告诉处理器取指令的正确地址,此时可以简单地停止流水线,避免以后从流水线中抹掉指令而造成的功耗浪费。

(2) 在读取物理寄存器的阶段(有可能在寄存器重命名阶段之后,也有可能是在发射阶段之后,这取决于处理器的架构,在后文会有详细介绍),如果此时读取到了寄存器的值,那么就可以对间接跳转类型的分支指令所预测的目标地址进行检查,如果发现目标地址预测错误了,那么就可以使用正确的地址开始重新取指令,此时仍旧需要将分支指令之后进入到流水线的指令从流水线中抹掉,这样并不是很容易实现,因为这些指令很可能已经进入了发射队列(Issue Queue)中,当然在发射队列中还存在正确的指令,因此需要对其中的指令进行选择性的清除,这部分内容在后文中会进行详细介绍。

(3) 在执行阶段,不管什么类型的分支指令都可以被计算出结果,此时可以对分支预测是否正确进行检查。当然,如果一条分支指令到这个阶段才对预测是否正确进行检查,当发现预测失败时造成的惩罚(penalty)也是最大的。

在执行阶段发现一条分支指令的预测失败时,在这条分支指令后面进入到流水线的指令都需要从流水线中抹掉,但是需要注意的是,由于超标量处理器多采用乱序执行(out-of-order)的方式,在此时的流水线中,还有部分指令是在这条分支指令之前进入流水线中的,这些指令可能位于发射队列(Issue Queue)中,也可能位于执行阶段,它们不应该受到分支预测失败的影响,可以继续执行,如何才能实现这样的功能呢?其实,指令之间本来的顺序在流水线的重排序缓存(ROB)中进行了记录(ROB 将在后面章节专门介绍),因此可以利用重排序缓存对分支预测失败时的处理器进行状态恢复,这部分内容将在第 10 章进行详细的介绍,此处进行简要的说明。

当在执行阶段发现了一条分支指令预测失败时,将这个信息记录在这条分支指令在 ROB 对应的表项(entry)中,并暂停流水线的取指令,但是让流水线继续执行,当这条分支指令变为流水线中最旧的指令时,表明在这条分支指令之前进入到流水线中的所有指令都已经离开了流水线,此时在流水线中的所有指令都处于分支预测失败(mis-prediction)的路径上了,可以将整个流水线中的指令全部抹掉,从正确的地址重新开始取指令来执行。这个方法最大的缺点就是需要分支指令在流水线中等待一段时间才能进行处理,并且在分支指令之前存在 D-Cache 缺失的 load 指令时,这个等待的时间就会很长,在这段时间内流水线都无法取新的指令来执行,这样就会造成分支预测失败时的惩罚(penalty)很大,降低了处理器的性能,这种使用 ROB 对处理器进行状态恢复的方法如图 4.50 所示。

虽然图 4.50 所示的方法在某些情况下会造成分支预测失败时的惩罚过大,但是这种方法很容易实现,是在硬件复杂度和执行效率之间的一个比较折中的方案,Intel 的很多处理器就采用了这种方法。

除了采用基于 ROB 的方法进行分支预测失败时的状态恢复,很多现代的处理器的处理器都采用了基于 Checkpoint 的方法进行状态恢复。所谓 Checkpoint,是指发现分支指令,并且在

分支指令之后的指令更改处理器的状态之前,将处理器的状态保存起来,这里所要保存的处理器状态主要是指寄存器重命名中使用的映射表(mapping table),当然还包括其他的一些信息,例如预测跳转的分支指令对应的下一条指令的 PC 值等。一般来说,当分支指令进入流水线的寄存器重命名阶段的时候,就需要进行这个状态保存的过程,将此时的映射表保存起来,不同形式的映射表对于硬件需求也是不同的。总体看起来,相比使用 ROB 的方式,使用这种 Checkpoint 的方式进行分支预测失败时的状态恢复,需要消耗更多硬件资源,但是这种方法能够快速地将处理器的状态恢复,并马上开始从正确路径取指令来执行,因此这种方法的执行效率更高。

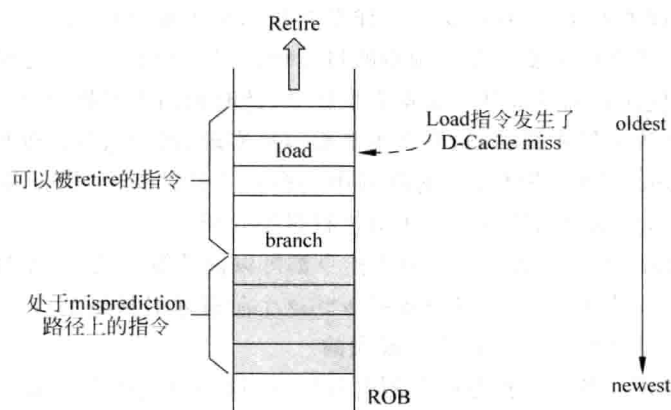


图 4.50 使用 ROB 对分支预测失败时的处理器进行状态恢复

相比于使用 ROB 的方法,使用 Checkpoint 方法进行状态恢复,还有一个重要的任务就是将流水线中所有处于分支预测失败(mis-prediction)路径上的指令都抹掉。但是,当分支指令到了执行阶段,发现分支预测失败的时候,此时流水线中仍有一部分指令并没有处于分支预测失败的路径上,这些指令不能够被抹掉,因此需要一种机制来正确地识别哪些指令处于这条错误的路径上,这可以通过对每一条分支指令进行编号来实现,所有在这条分支指令后面进入到流水线的指令都会得到这个编号,直到遇到下一条分支指令为止,这个过程如图 4.51 所示。

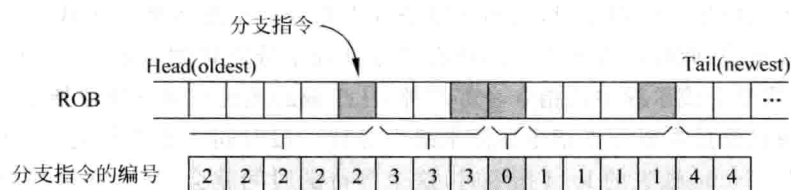


图 4.51 对每一条分支指令都使用一个编号

在图 4.51 中,为每条分支指令都分配一个编号,这样可以很方便地识别出哪些指令位于这条分支指令的后面,当发现分支预测失败时,就可以马上利用这个编号,对流水线中的指令有选择地进行抹掉,而不需要等到这条预测错误的分支指令变为流水线中最旧的指令。分支指令的编号个数决定了最多可以在流水线中存在的分支指令个数,这可以通过下面的方法进行估算:假设处理器最多支持 128 条指令存在于流水线中,按照每五条指令存在一

条分支指令来计算,最多会有 $128/5=26$ 条分支指令存在于流水线中,也就是说,分支指令的编号需要 26 个,则这个编号的位数需要 5 位才可以满足要求($2^5=32$)。

所有在流水线中的分支指令都会被分配一个编号值,这些编号值都会保存在一个 FIFO 中,这个 FIFO 称为编号列表(tag list),它的容量等于处理器最多支持的分支指令的个数,例如处理器最多支持 26 条分支指令存在于流水线中,则编号列表的容量就是 26 个,一旦这个列表满了,就不能再向流水线中送入分支指令。因此,如果再解码出分支指令,就会暂停解码阶段之前的流水线,直到编号列表中有空余的空间为止。

当然,上面的方法只是给出了一种设计的思路,在具体实现时,有很多种方法都可以对分支指令的编号进行分配和回收,下面给出一种一般性的设计方法。这种方法使用两个表格,一个表格用来存放所有没有被使用的编号值,这个表格称为空闲的编号列表(free tag list);另一个表格用来存储所有被使用的编号值,即上面所说的编号列表(tag list)。每次在解码阶段发现一条分支指令时,就从空闲的编号列表(free tag list)中送出一个编号值,这个编号值会被写到编号列表(tag list)中,在此之后被解码的所有指令都被附上这个编号值,直到遇到下一条分支指令被解码为止。

这两个编号列表(free tag list 和 tag list)本质上都是 FIFO,一旦在流水线的执行阶段发现一条分支指令预测错误了,就会使用编号列表中这条分支指令对应编号值,以及它之后的所有编号值,将流水线中对应的指令进行抹掉,同时编号列表中这些用来抹掉流水线的编号值会被放回到空闲的编号列表中,供后续进入流水线的分支指令使用,它们的工作流程可以用图 4.52 来表示。

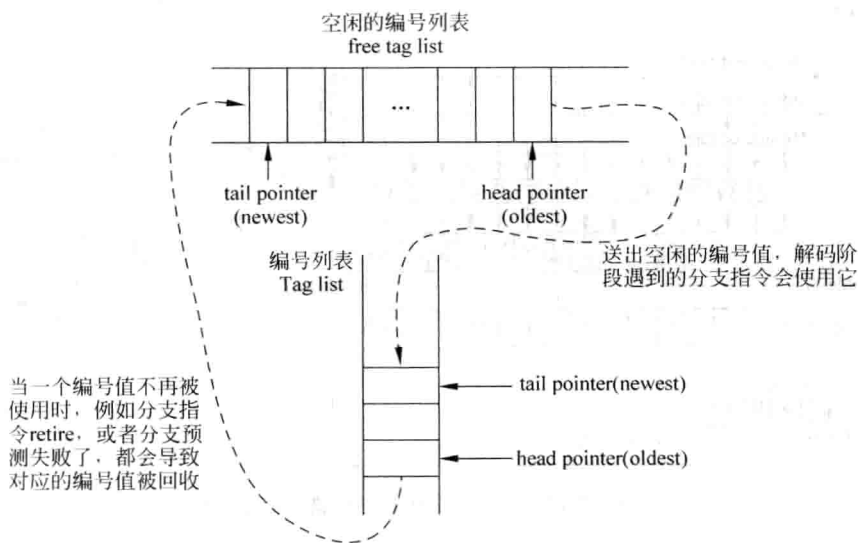


图 4.52 使用 free tag list 和 tag list 配合进行工作

前面说过,在解码阶段也会对分支指令的预测结果进行检查,如果在这个阶段发现了预测错误,此时流水线中处于解码阶段之前的所有指令都应该从流水线中抹掉,然后从正确地址开始取指令。当然,这条分支指令也仍然会被分配编号并写到编号列表中,因为此时并不确定它之前是否存在预测失败的分支指令,万一在随后的某个时间,在流水线的执行阶段发现了一条预测错误的分支指令,那么刚刚在解码阶段被纠正的分支指令也就处在了分支预

法接受的。

使用分支指令的编号列表对流水线中的其他部件,例如发射队列(Issue Queue),进行处理的过程和图 4.53 是类似的,虽然指令在发射队列中的排布并没有什么顺序,但是仍然可以使用上述的方法对其中相关的指令进行抹掉。当然,对于处于流水线中发射(issue)阶段之后的指令,也仍然可以使用这种方法。

将流水线中相关的指令抹掉之后,就需要使用 Checkpoint 对处理器的状态进行恢复,主要是对寄存器重命名阶段的映射表(mapping table)进行恢复,关于这部分内容将在后文进行介绍。

其实,并不是一定需要在一个周期之内将重排序缓存(ROB)和发射队列(Issue Queue)中的相关指令都抹掉,因为在流水线的执行阶段发现一条分支指令预测错误时,需要从正确的地址开始取指令,这些新取出来的指令需要经过流水线的好几个阶段才可以到达发射阶段(在流水线的发射阶段需要使用重排序缓存和发射队列),只要在新指令到达这个阶段之前,重排序缓存和发射队列(还包括发射阶段之后的其他流水段)中的相关指令被抹掉,那么流水线就可以使用到正确的状态,不需要暂停了。正因为如此,可以采用一种折中的方法,每周只从分支指令的编号列表(tag list)中广播一个(或几个)编号值,这样当编号列表中的编号值不是很多时(大部分时候都是如此),使用几个周期就可以将流水线中相关的指令抹掉,此时并不会影响流水线的执行;只有在少数情况下,当编号列表中的编号数量很多时,才会需要花费多个周期来抹掉流水线中的相关指令,此时就需要新指令在到达流水线的发射阶段的时候进行等待,直到发射阶段以及之后的流水段中,所有相关的指令都被抹掉了,并且完成了状态恢复,才允许新指令进入到流水线的发射阶段。

需要强调的是,上面的这种使用分支指令的编号来抹掉流水线中相关指令的方法,一定是对每一条分支指令都分配一个编号值,这样才能在分支指令预测正确时,将属于这个编号值的所有指令置为非推测(non-speculative)的状态,而在分支指令预测失败时,抹掉流水线中所有处于分支预测失败(mis-prediction)路径上的指令。如果对某条分支指令没有分配编号值,那么在执行阶段对这条分支指令是否预测正确进行检查时,就无法正确地识别出哪些指令处于这条分支指令的路径上,这样上述的方法就无法正常工作了。那么,什么时候对分支指令分配这个编号值呢?如果在流水线的取指令阶段就进行这个编号的分配,由于此时只有 PC 值,所以对分支指令的识别是推测的(speculative),不能够保证将所有的分支指令都分配编号。只有在解码阶段,对分支指令进行编号值的分配是最合适的,因为在这个阶段可以确定地识别出哪些指令是分支指令,此时就可以确保对每一条分支指令都分配一个编号值。

在每周期执行多条指令的超标量处理器中,上述对分支指令分配编号值的过程会遇到新的问题。对于一个 N -way 的超标量处理器来说,最坏的情况是在一个周期内解码的 N 条指令都是分支指令,那么空闲的编号列表(free tag list)就需要在一个周期之内提供 N 个编号值,同时编号列表(tag list)也需要在一个周期内可以被写入 N 个编号值,这样就需要使用多端口的 FIFO,这对于芯片的面积和功耗是有负面影响的。但是在绝大部分情况下,每周期解码的指令中可能最多只有一两条分支指令,因此 FIFO 的大部分端口都是空置状态,并没有被有效地利用,为这种极少见的情况设置复杂的硬件,不是 RISC 处理器的风格。因此在处理器内部可以进行一个折中的约定,在流水线的解码阶段每周期最多只能处理一

条分支指令,如果在解码阶段,发现从指令缓存(Instruction Buffer)中读出的 N 条指令中存在多于一条的分支指令(指令缓存是位于取指令和解码阶段之间的一个缓存,保存着取指令阶段取出的所有指令),那么第二条分支指令及其后面的所有指令在本周期就不能够进入解码阶段,它们会被延迟到下一个周期。对于大部分程序来说,分支指令都并不密集,因此这种方法不会对性能造成太大的负面影响。

对于分支预测结果的检查,还需要考虑一个问题:当分支指令到了执行阶段得到实际的结果,并对分支预测是否正确进行检查时,如何才能知道这条分支指令在之前的预测值?

要想在流水线的后续阶段得到分支指令的预测值,就需要在流水线的解码阶段,将每条分支指令的预测值保存起来,例如,可以将每条分支指令的预测值写到一个缓存(buffer)中,每条分支指令在缓存中的地址都会随着分支指令在流水线中流动,这样当分支指令到达流水线的执行阶段,进行分支预测是否正确的检查时,就可以从缓存中读取到这条分支指令在之前的预测值了。这种设计理念在实际的处理器设计中可以再进行优化,只将那些方向预测是跳转的分支指令保存在缓存中,如图 4.54 所示。

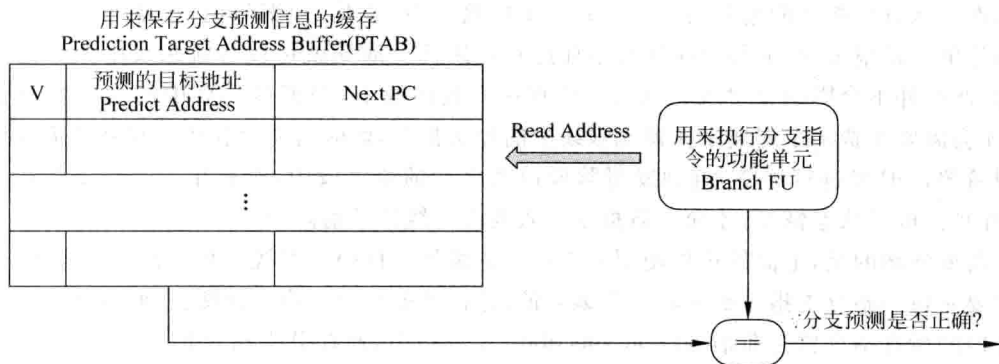


图 4.54 使用缓存来保存那些预测跳转的分支指令

在图 4.54 中,对方向预测是跳转(taken)的分支指令进行保存的地方称为 PTAB (Prediction Target Address Buffer),它主要由三部分组成。

(1) Valid,表示 PTAB 中某个表项(entry)是否被占用,当一条方向预测为跳转的分支指令写入到 PTAB 中时,会将这一位置为 1,表示这个表项被占用;当这条分支指令到达流水线的执行阶段,完成了分支预测是否正确的检查时,这个表项就可以被释放了,因此将这一位清零。

(2) Predict Address,分支指令被预测的目标地址。

(3) Next PC,方向预测为跳转的分支指令的下一条指令的 PC 值,如果发现这样的分支指令预测错误,直接使用这个值就可以作为正确的取指令地址。

一条分支指令如果被预测发生跳转,那么会将它的信息写到 PTAB 中,同时这条分支指令在 PTAB 中的地址也随着分支指令在流水线中流动,这样,当这条分支指令到达流水线的执行阶段时,就可以使用这个地址,直接从 PTAB 中找到对应的预测信息了。

由于在超标量处理器中,并不能保证对所有的分支指令都进行了分支预测,当然也不能保证所有预测的结果都是正确的,所以在流水线的执行阶段得到分支指令的实际结果时,会

有下面的几种情况。

(1) 实际结果是不发生跳转,并且在 PTAB 中没有找到对应的内容,那么就表示一条分支指令被预测不发生跳转,实际也没有发生跳转,因此预测正确;

(2) 实际结果是不发生跳转,但是在 PTAB 中找到了对应的内容,那么就表示一条分支指令被预测发生跳转,实际却没有发生跳转,因此预测错误,需要使用 Next PC 的值作为正确的目标地址;

(3) 实际结果是发生了跳转,但是在 PTAB 中没有找到对应的内容,那么就表示一条分支指令被预测不发生跳转,实际结果却发生了跳转,因此预测错误,需要使用实际计算出来的目标地址;

(4) 实际结果是发生跳转,并且在 PTAB 中找到了对应的内容,那么就表示一条分支指令被预测发生跳转,实际也发生了跳转,此时需要将实际计算出来的目标地址和 PTAB 中记录的预测值进行比较,如果相等,则表示预测正确,如果不相等,则表示预测错误,需要使用实际计算出来的目标地址。

由于 PTAB 只需要保存所有预测跳转的分支指令,而不需要将全部分支指令的信息进行保存,因此这样可以减少 PTAB 的容量,在这种情况下,写 PTAB 的过程可以在流水线的取指令阶段完成,只要在取指令阶段预测到一条发生跳转的分支指令,就将其写到 PTAB 中。那么,采用这种方式后,有没有可能出现将一条非分支指令预测为发生跳转的情况呢?

如果在程序中存在自修改(self-modifying)的代码,可能会将一条分支指令修改为普通的计算指令,如果此时没有将分支预测器(branch predictor)中的内容清空,那么就有可能出现将这条计算指令预测为发生跳转的错误情况了。但是,这个错误在流水线的解码阶段就会被检测到,它所占据的 PTAB 中的内容也会被释放掉。其实在实际应用当中,程序中的自修改(self-modifying)代码执行完成后,都需要将分支预测器(branch predictor)、I-Cache 等部件中的内容都进行清空,这样才能够保证流水线可以正确地执行被修改过的指令。

更多关于分支预测失败时,处理器状态恢复的内容,可以参见参考文献中的论文^{[19][20]}。

4.5 超标量处理器的分支预测

到目前为止,本章所讲述的分支预测都是对一条指令(也就是一个 PC 值)进行分支预测,对于普通的处理器(每周只取一条指令),这样是没有问题的,但是对于超标量处理器来说,在取指令时给出一个地址,会从 I-Cache 中取出多条指令,这些取出的指令组成了一个指令组(fetch group),处理器会自动根据指令组中的指令个数,调整取指令的地址,用来进行下个周期的取指令。因此超标量处理器中的取指令地址并不是连续的,每次增加的值等于指令组的字长,送入到 I-Cache 中取指令的地址其实只是指令组中第一条指令的地址而已。如果此时仍旧只是使用取指令时的地址(也就是 PC 值)进行分支预测,那么就相当于只是对指令组中的第一条指令进行了分支预测,而指令组中后面的指令根本就没有顾及到,例如每周取指令的个数为 4 时,就相当于只对 1/4 的指令进行了分支预测,这样显然降低了分支预测的准确度,使超标量处理器无法从分支预测中获得太大的好处。

当然,如果对取指令的过程进行限制,例如对于一个 4-way 的超标量处理器来说,使每周取出的指令位于四字对齐的边界之内,那么这样就可以使用它们的公共地址(PC[31:

4)]来寻址分支预测器,而对于分支预测器来说,它只需要记录下每组四字对齐的指令中,第一个预测跳转的分支指令的信息就可以了。因为在大多数情况下,每四字对齐的四条指令中最多也就存在一条分支指令,所以使用这种方法不会有什么问题,只不过需要注意的是,在 BTB 中需要记录下分支指令在四条指令中的位置,避免有些时候错误地使用它的结果,如图 4.55 所示。

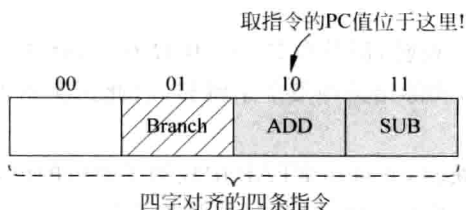


图 4.55 本周期取出的指令中不包括分支指令

在图 4.55 中,当前周期取出的指令中是不包括分支指令的,所以不应该使用这条分支指令的预测信息,这需要在 BTB 中记录下分支指令的偏移值。例如在图 4.55 中就需要记录下“01”这个偏移值,当进行分支预测时,如果发现分支指令不在当前取指令的范围之内,那么就不会使用它的预测值了。

不过,当四字对齐的四条指令中存在多于一条的分支指令时,它们就会互相干扰,从而影响分支预测的准确度,不过在大部分的程序中,分支指令出现的频率都不会这么高的。

如果对于一个 4-way 的超标量处理器来说,每周期取出的指令可以不局限于四字对齐的边界之内,那么要达到最理想的效果,就需要对一个周期内取出的所有指令都进行分支预测,将第一个预测跳转(taken)的分支指令的目标地址作为下个周期取指令的地址,如图 4.56 所示。

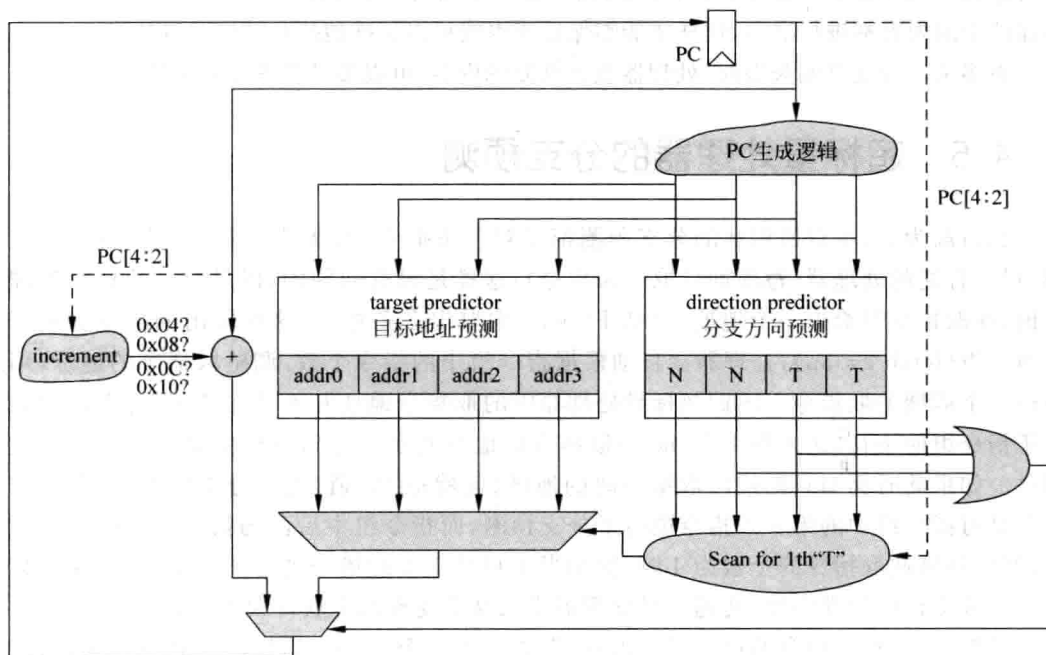


图 4.56 对一个周期内取出的所有指令都进行分支预测

要对指令组(fetch group)中所有的指令都进行分支预测,就需要得到所有指令的地址(也就是PC值),而取指令阶段使用的PC值只给出了指令组中第一条指令的PC值,需要使用三个加法器来实现 $PC+4$ 、 $PC+8$ 和 $PC+12$ 的功能,这样就可以得到本次指令组中所有指令的PC值了。但是由于在一个周期中,取出的指令都在一个Cache line之内,例如对于32B的Cache line来说,一个周期取出的所有指令的PC值中,[31:5]是一样的,所以只需要对地址的[4:2]部分使用上述三个加法器即可。图4.56所示的这种对多条指令同时进行分支预测的方法,对分支指令的方向预测和目标地的预测是同时进行的,这样可以保证尽快地得到分支预测的结果,但是在实际当中却很难被接受,为什么呢?这是由于对目标地址的预测来说,需要在一个周期内提供四个PC值对应的目标地址,这就相当于需要BTB支持四个读端口,即使可以采用交叠(interleaving)的方式来避免真正的多端口,但是考虑到在使用过程中,最多只会使用其中的一个端口的值,所以这种方式对于硬件的利用效率是很低的。

如果可以在分支指令的方向预测完毕之后,利用它的结果信息再进行目标地址的预测,可以避免对BTB等部件的多端口需求了,这种方法对于方向预测和目标地址预测这两部分的访问是串行的,但是这样做之后,分支预测很可能就无法在一个周期内完成,这会导致流水线无法连续地“猜测”每周取指令的地址,会造成处理器的性能有一些损失,但是却可以使硬件的复杂度得以降低,因此也不失为一种可以接受的折中方法。

其实,对于目标地址的预测,大部分情况下可能并不需要这样复杂。对于RISC指令集的处理器来说,例如MIPS,程序中绝大部分时候出现的分支指令都是直接跳转(PC_relative)类型,它们的目标地址可以很快地被计算出来,因此对于这种类型的分支指令来说,其实可以不必进行目标地址的预测,而是直接在取指令阶段,只要指令被取出来,就马上进行目标地址的计算,这样虽然也不能使分支预测在一个周期内完成,但是它所耗费的时间不一定比上述串行访问所需要的时间要长,而且还可以得到一个正确的目标地址,因此比较来看,这种方法要更优一些。当然,要实现这样的功能,最好能够在指令进入I-Cache之前,进行预解码,例如可以将哪些指令是分支指令的这种信息标记出来,这样可以使指令从I-Cache取出来的时候,马上识别出分支指令,从而快速地进行目标地址的计算,MIPS的74kf处理器就采用了这样的方法,对分支指令的目标地址进行快速的计算。当然,如果遇到的分支指令是间接跳转类型的,而且不是Return指令,那么此时就没有办法对目标地址进行预测了。

对于分支指令的方向预测来说,在超标量处理器中,也需要每周对多条指令进行方向预测,从而找到第一条预测跳转的指令。对基于局部历史的分支预测方法来说,需要PHT和BHT都支持多个读端口,这可以通过交叠(interleaving)的方式来模拟实现多端口。但是对基于全局历史的分支预测方法来说,不是简单地将PHT扩展为多个读端口就可以的,因为在一个周期内进行分支预测的多条指令对应的GHR有可能是不一样的,例如一个周期中包括两条分支指令时,这两条分支指令对应的GHR就是不同的,这需要进行特殊的处理,参考文献中的论文给出了一些解决方法^{[21][22]}。

交叠(interleaving)方式是在超标量处理器中经常使用的一种结构,这种结构其实和上文中讲述的多端口Cache是类似的,都可以利用单端口存储器来模拟多端口的结构,举例来说,对于PHT来说,可以采用图4.57所示的结构来实现四个读端口。

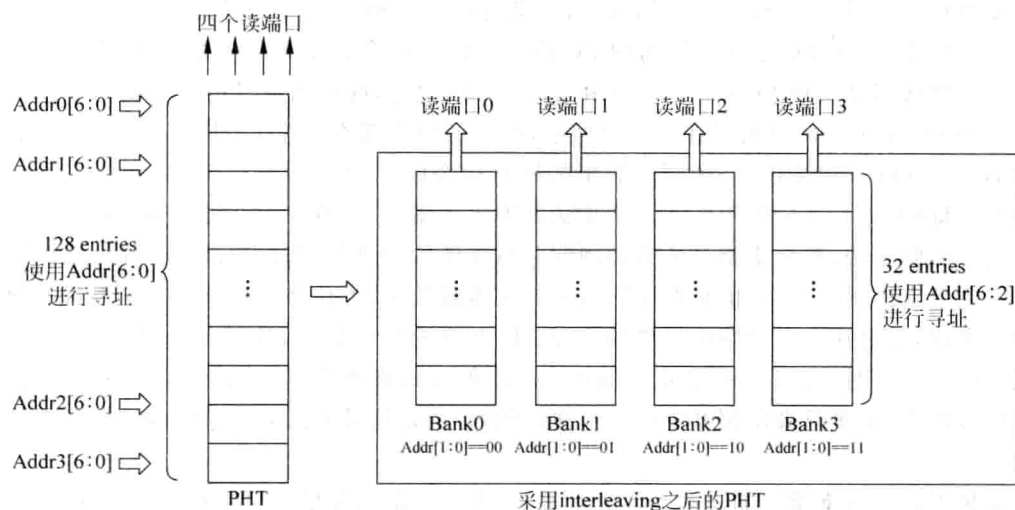


图 4.57 使用交叠(interleaving)方式来实现多端口 PHT

在图 4.57 中,原本的 PHT 的容量是 128,使用 7 位的地址 $\text{Addr}[6:0]$ 进行寻址,当采用交叠(interleaving)结构之后,PHT 由四个 bank 组成,每个 bank 都是一个单端口的存储器,它的容量是 32,此时可以根据地址 $\text{Addr}[1:0]$ 找到对应的 bank(也就是 $\text{Addr}[1:0]$ 可以作为每个 bank 的片选信号),然后使用其余的地址 $\text{Addr}[6:2]$ 来找到 bank 内相应的内容。举例来说,对于地址 $\text{Addr}[6:0]=0011110$,由于它的 $[1:0]$ 是 10,会寻址到 bank2,地址中其余的部分 $[6:2]$ 是 00111,用来查找 bank2 中的内容,这样就可以找到对应的值了。不过需要注意的是,由于寻址 PHT 的地址可能是由哈希(Hash)运算产生的,这种运算要保证每周寻址 PHT 的四个地址会落在不同的 bank 上,这样才可以保证不出现 bank 冲突,从而造成在一个周期内不能够将四条指令全部的分支预测值进行读出,如何才能够使哈希运算保证这个功能呢? 其实最简单的方法就是将每条指令 PC 值中的 $[3:2]$ (注意 PC 是需要字对齐的)作为寻址 PHT 地址的低位部分,这样就可以保证四个地址会寻址到 PHT 中的四个不同的 bank 了。

图 4.57 所示的这种交叠(interleaving)的设计方式,使用几个单端口的存储器组成了多端口的功能,避免了真正使用多端口存储器而对芯片的面积、功耗和延迟造成负面的影响,其实,在超标量处理器内部,这种交叠(interleaving)结构的存储器是经常被使用的,大部分多端口的功能部件都需要使用这种结构,例如重排序缓存(ROB)、发射队列(Issue Queue)和指令缓存(Instruction Buffer)等部件,这些部件将在后文进行详细的介绍。